

From Prompt to Panel: Enhancing Character Consistency in AI-Generated Comics via Textual Inversion

Shehij Raina

BSc (Hons) Computer Science
The University of Bath
May 2025

From Prompt to Panel: Enhancing Character Consistency in AI-Generated Comics via Textual Inversion

Submitted by: Shehij Raina

Copyright

Attention is drawn to the fact that copyright of this dissertation rests with its author. The Intellectual Property Rights of the products produced as part of the project belong to the author unless otherwise specified below, in accordance with the University of Bath's policy on intellectual property (see https://www.bath.ac.uk/publications/university-ordinances/attachments/Ordinances_1_October_2020.pdf).

This copy of the dissertation has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with its author and that no quotation from the dissertation and no information derived from it may be published without the prior written consent of the author.

Declaration

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

Abstract

This dissertation presents a text-to-image pipeline for generating narratively coherent comics with visually consistent characters across panels, from a single text prompt, achieved by fine-tuning a diffusion model using textual inversion techniques. We also introduce a novel semantic loss term to our textual inversion algorithm and report on its efficacy in promoting consistency across key visual character traits such as hair colour and clothing, while adapting to diverse prompts and storytelling contexts.

The results of this project were evaluated by human participants in a study, rating comics on their character consistency, narrative coherence, adherence to the prompt and image quality. Comics generated using our pipeline consistently outperformed those generated by the baseline Stable Diffusion model - with those incorporating the semantic loss term receiving a mean score of 8.57 across all the evaluation criteria, compared to 4.67 for the Stable Diffusion baseline 5.3. These results highlight the effectiveness of our approach in producing high-quality, AI-generated comic strips with enhanced narrative coherence and visual consistency.

Contents

1	Introduction	1
1.1	Artificial Intelligence in Comic Creation	1
1.1.1	Notable Developments	1
1.2	Problem Description	2
2	Background	5
2.1	Introduction to Diffusion Models	5
2.1.1	Forward Process	6
2.1.2	Reverse Process	6
2.1.3	Training	6
2.1.4	Other Concepts	7
2.2	Stable Diffusion	8
3	Related Work	10
3.1	Textual Inversion	10
3.2	Other Relevant Work	12
4	Methodology	14
4.1	Input Processing	14
4.2	Training Textual Inversion	18
4.2.1	Standard Training	19
4.2.2	Experimental Semantic Loss	19
4.3	Panel-wise Image Generation	21
4.4	Style Transfer	23
4.4.1	Introduction to LoRA	23
4.4.2	Dataset	24
5	Results	28
5.1	Output Evaluation	28
5.2	User Study	31
6	Conclusions	37
6.1	Limitations and Future Work	37
6.2	Final Remarks	38
	Bibliography	39
A	Code	42

A.1 File: ConsistentCharacterComics.py	43
A.2 File: StyleTransfer.py	53
B User Study	57

List of Figures

1.1	Table depicting the vision-language tasks from the LoCU framework. (Sourced from: awesome-comics-understanding)	2
1.2	An image generated by the Midjourney V6.1 API (using Midjourney.FM) with the prompt "A comic strip with multiple panels showing a superhero with a red cape and mask making friends with people around the world."	3
2.1	Image depicting the forward and reverse processes in diffusion models (Sourced from CVPR)	5
3.1	Textual Inversion being used to learn a concept (Sourced from: <i>An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion</i>)	10
3.2	Textual Inversion Training - A description string containing the new pseudo-word is tokenized and then converted into continuous vector representations. The embedding vector $\mathbf{v}^*$ associated with the pseudo-word S^* , is what is to be learnt. (Sourced from: <i>An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion</i>)	11
4.1	A diagram illustrating the complete pipeline	15
4.2	Training images for textual inversion	17
4.3	Comic panels generated by the diffusion model after standard textual inversion.	22
4.4	A selection of images from the Studio Ghibli Dataset	24
4.5	Distribution of RGB values across all pixels in the Studio Ghibli Dataset	24
4.6	The top 20 most commonly used colours across the Studio Ghibli Dataset	25
4.7	Mean Colour of the Studio Ghibli Dataset	25
4.8	The final comic produced by our pipeline for the prompt "Sarah visits the library".	27
5.1	Top Left: Standard Textual Inversion before style transfer. Bottom Left: Standard Textual Inversion after style transfer. Top Right: Textual Inversion employing semantic loss before style transfer. Bottom Right: Textual Inversion employing semantic loss after style transfer.	29
5.2	Comic A: Comic generated by Stable Diffusion 2-1 for the prompt "A comic strip for "Sarah visits the library". Retrieved from HuggingFace.	30
5.3	Comic B: Comic generated by an online image generator for the prompt "A comic strip for "Sarah visits the library". Retrieved from pixlr	30
5.4	Comic C: Our pipeline with standard textual inversion	30
5.5	Comic D: Our pipeline with textual inversion employing semantic loss	31
5.6	Boxplot of the four comics evaluated on "Character Consistency".	32
5.7	Boxplot of the four comics evaluated on "Story Coherence"	32

5.8	Boxplot of the four comics evaluated on "Adherence to Prompt"	32
5.9	Boxplot of the four comics evaluated on "Image Quality"	32
5.10	Heatmap of Comics A, B, C and D evaluated on the above four metrics. The darker colours (depicting higher values) are concentrated at the bottom of this figure, showing that Comics C and D (produced by our pipeline) vastly outperform diffusion models that generate the whole comic altogether. . . .	33

List of Tables

5.1	Summary Statistics	31
5.2	Percentage of Ratings Below 4	33
5.3	Mean of all four criteria for each comic	33
5.4	Average ratings by metric for each comic.	34

Acknowledgements

I would like to thank my supervisor, Dr. Deblina Bhattacharjee, for her unwavering support, insightful guidance, and exceptional expertise throughout the course of this research. Her thoughtful feedback and encouragement were instrumental in shaping the direction of this work and in overcoming challenges along the way.

I am also deeply grateful to the faculty and staff of the Department of Computer Science at the University of Bath for their continuous support and for providing access to essential resources, including the Hex GPU Cloud, which played a vital role in the completion of this project.

Chapter 1

Introduction

Visual media has long served as a powerful tool for human communication and storytelling. From prehistoric cave paintings to modern digital art, visual narratives transcend language and cultural barriers, making them a universal form of expression (Oktaviana et al., 2024).

Comics, a popular form of visual storytelling, have proven to be one of the most enduring forms of media in history, offering unique ways to explore complex themes and emotions, as well as being a tool for cultural reflection and critique (Duncan and Smith, 2009). As Scott McCloud argues in his seminal work "Understanding Comics: The Invisible Art" (McCloud and Martin, 1993), the combination of images and words in comics produces a synergistic effect that allows the narrative to unfold at a pace and depth unattainable in either medium alone.

Moreover, Research indicates that 65 percent (ca. Bradford, 2011) of the general population are visual learners, meaning they retain information better when it is presented visually. As a result, comics serve as a powerful medium for presenting complex ideas in an accessible way, further reinforcing their effectiveness as educational tools (Pange, 2023).

1.1 Artificial Intelligence in Comic Creation

Artificial Intelligence has emerged as a transformative force in the creative industries. It is being employed in various stages of comic production, from conceptualization to illustration, and in a variety of applications that leverage Generative AI to produce comics automatically or semi-automatically (Kumar, Babu and Srinivasan, 2024).

1.1.1 Notable Developments

One of the most notable developments is ComicGAN (Proven-Bessel, Zhao and Chen, 2021), a text-to-image generative adversarial network tailored to comics, which produces comic strips out of text descriptions, *or dialogue* - by using automatic text description generation, addressing a key challenge of abstract semantic alignment between comic imagery and text. It also makes use of specialized encoders adapted to handle the stylization gap between photorealistic visual features and artistic, exaggerated comic features (like line art and flat colours).

Yang et al. (2021) introduce a pioneering video-to-comic pipeline to automate the conversion of film into comic-book pages. The process uses the video's subtitles to identify informative key-frames to be stylized to resemble comic art. The layout is designed to be visually appealing

by analysing the semantic content of the images, their regions of interest and relationships between them.

Aside from their production, Artificial Intelligence has also made significant strides in understanding comics and, grasping their visual context, character interactions, and narrative structure. A Comics Understanding Survey by Vivoli et al. (2025) highlights the challenges faced in interpreting comics, such as non-linear storytelling and diverse artistic styles. It also introduces the Layer of Comics Understanding (LoCU) framework, categorizing and ranking vision-language tasks like image classification, object detection, text-to-image generation etc. based on their input and output modalities and dimensions, to guide future research in this area 1.1.

Layer	Category	Task	Input	Output
0 (Sec.5)	View (Sec. 5.1)	Basic Image Viewing (BIV)	Text Command	Image Display
1 (Sec.6)	Tagging (Sec. 6.1)	Image Classification (I-CLS)	Image	Tag
		Emotion Classification	Comic Panels/Images	Emotion Labels
		Action Detection	Multiple Panels	Tag
	Augmentation (Sec. 6.2)	Page Stream Segmentation (PSS)	Images	Tags sequence
		Image Super-Resolution (ISR)	Image	Image
		Style Transfer (ST)	Image	Image
2 (Sec.7)	Grounding (Sec. 7.1)	Vectorization	Comic Panels/Images	Vector Image
		Depth Estimation	Comic Panels/Images	Depth Map
		Object Detection	Tag/s + Image	Bounding Boxes
	Analysis (Sec. 7.2)	Character Re-identification (Character ID)	Multiple Panels	Tag
		Grounding (IG)	[Prompt] + Image	Bounding Boxes
3 (Sec.8)	Segmentation (Sec. 7.3)	Character-Balloon Association (Speaker ID)	Character + Balloons	Tag
		Dialog transcription	Image	Text
		Translation	Image	Text
	Retrieval (Sec. 8.1)	Instance Segmentation (IS)	[Prompt] + Image	Segments
		Image-Text Retrieval (IR)	Text	Image
4 (Sec.9)	Modification (Sec. 8.2)	Text-Image Retrieval (TR)	Image	Text
		Composed Image Retrieval (CIR)	Text + Image	Image
		Image Inpainting (II)	Text + [prompt] + Image	Image
	Understanding (Sec. 9.1)	Image Editing via Text (IET)	Text + Image	Image
		Visual Entailment (VE)	Image + Text	Tag
5 (Sec.10)	Generation (Sec. 10.1)	Visual Question Answering (VQA)	Image + Text	Text
		Visual Dialog (VisDial)	Image + Dialog + Text	Text
		Visual Reasoning (VR)	Image + Text	Text
	Synthesis (Sec. 10.2)	Image-2-Text Generation (I2T)	Image	Text
		Grounded Image Captioning (GIC)	Image	Text + Bbox
		Text-2-Image Generation (T2I)	Text	Image
		Scene Graph Generation for Captioning	Comic Panel	Scene Graph
		Sound Generation from Single Panel	Single Comic Panel	Sound/Audio
		3D Model Generation from Images (3DGI)	Collection of Images	3D Model
		Video Generation from Text (VGT)	Complex Long Text	Video
		Narrative-Based Complex Scene Generation (NCSG)	Detailed Narrative Text	Series of Images

Figure 1.1: Table depicting the vision-language tasks from the LoCU framework. (Sourced from: awesome-comics-understanding)

These developments underscore the richness and rapid growth of research at the intersection of Comics and Artificial Intelligence. In the following section, I build upon this foundation by addressing a specific gap within this landscape, outlining the problem this dissertation aims to solve.

1.2 Problem Description

The characters of a story make up its core. This also applies to the narrative medium of comics. Within each panel, it is the characters who animate the storyline and serve as the primary agents of reader engagement and emotional resonance. Tseng, Laubrock and Pflaeging (2018),

in *Comics and Graphic Novels: Digital, Multimodal, and Cognitive Methods* discuss how cohesion in comics is achieved through the consistent presentation of characters, objects, and settings. They state that cohesive chains bind together information about recurring elements across images and texts, and this consistency is vital for maintaining narrative coherence and guiding readers in understanding the storyline.

Take a look at the following Figure 1.2 generated by the text-to-image model Midjourney V6.1. The prompt text provided as an input for this generation was "A comic strip with multiple panels showing a superhero with a red cape and mask making friends with people around the world."



Figure 1.2: An image generated by the Midjourney V6.1 API (using Midjourney.FM) with the prompt "A comic strip with multiple panels showing a superhero with a red cape and mask making friends with people around the world.".

The generated comic strip highlights several limitations inherent in contemporary image generation models, particularly in their application to comic creation. Apart from the presence of meaningless text, one of the most prominent issues is the lack of character consistency across panels. In the example produced, the central "superhero" figure exhibits noticeable variation in physical attributes: appearing with a male physique in one panel and a female one in another. Furthermore, defining visual characteristics from the prompt - "a red cape and mask" are inconsistently rendered or entirely omitted in some panels. These inconsistencies disrupt the visual continuity of the story, which hinders effective storytelling.

Such variation can be attributed to the underlying mechanics of diffusion models, which generate images by iteratively denoising a random distribution of pixels. While this process enables high levels of detail and realism, it also makes maintaining long-range spatial consistency, which is necessary for keeping the character visually identical across the image, particularly challenging.

Variations in pose, lighting, and composition may further exacerbate these difficulties, leading to deviations in character design within a single image.

This problem of character consistency persists even when trying to generate individual comic panels, one at a time. The same description of physical features may lead to the model generating vastly different characters, if not given extremely precise and repeatable instructions. This occurs because most image generation models operate independently on each prompt without memory or awareness of previously generated outputs. Their generative process includes a degree of stochasticity, which, while fostering creativity and variety, undermines their ability to replicate a specific character over multiple generations.

This limitation presents a considerable obstacle for artists and storytellers who wish to use image generation models for creating comics, as it hinders the development of coherent narratives, and leads to the end product being far from the creator's vision, rather than being personalised to their tastes.

The objective of this project is to investigate approaches to improving character consistency in AI-generated comics, thereby ensuring that they are not only visually appealing but also narratively coherent. A possible solution to this problem is to train a generative model to associate a particular word with a consistent visual representation of a character, and then use this word to generate the panels of the comic with consistent renderings of this character. This can be achieved through the process of textual inversion. This dissertation proposes a pipeline for generating comics with consistent characters by employing textual inversion to encode unique character identities into learned embeddings. Additionally, the project experiments with conditioning the embeddings for this newly-added character token on the textual description of the character (along with the training images) and evaluate whether this dual-input strategy further enhances stability and recognizability of characters throughout a comic sequence.

Chapter 2

Background

2.1 Introduction to Diffusion Models

Diffusion models are a class of generative models that have gained significant attention in recent years, particularly in the field of image generation. These models are based on the idea of progressively adding noise to an image and then reversing this process to recover the original image (Ho, Jain and Abbeel, 2020). This reverse process, known as "denoising", allows the model to generate new images by learning how to reverse the diffusion process from random noise to a structured image. Diffusion models are fundamentally different from other generative models, like GANs (such as ComicGAN), which use a two-network structure to generate images through adversarial learning involving a "Generator" (which produces fake images) and a "Discriminator" (which learns to distinguish between real and fake images) (Goodfellow et al., 2014). The core principle of diffusion models involves a forward-diffusion process and a reverse-denoising process.

In the forward process, noise is gradually added to an image over a series of steps, making it more and more distorted until it becomes indistinguishable from random noise. While, In the reverse process, a model is trained to learn how to gradually remove the noise and recover the original image. This denoising process involves using a neural network that tries to predict the noise added at each step of the reversal process, based on the noisy input it receives.

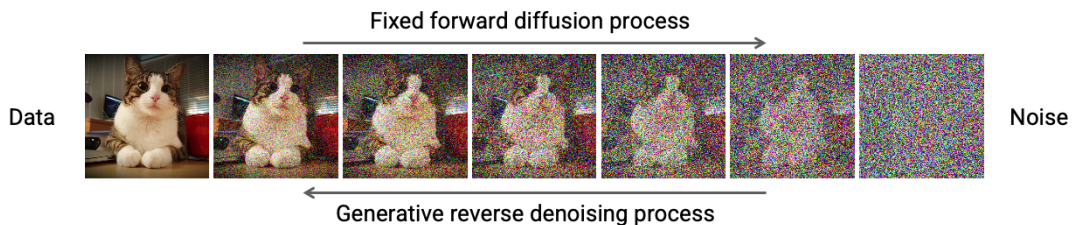


Figure 2.1: Image depicting the forward and reverse processes in diffusion models (Sourced from CVPR)

To better understand how diffusion models work, let's break down these core processes:

2.1.1 Forward Process

The forward diffusion starts with a clean image and applies a sequence of noisy transformations, in a Markov chain-like process, where each state x_t depends only on the previous state x_{t-1} , with Gaussian noise added gradually at each step making the image progressively less recognizable. The rate at which this noise is added depends on the chosen noise scheduling strategy. Mathematically, the forward process can be represented as:

$$x_t \sim \mathcal{N}(x_t | \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

Where x_t represents the image at time step t (for instance, x_0 is the original image), β_t is a small noise coefficient that controls the noise addition. The parameter β_t is typically scheduled to increase over time to ensure a smooth transition from a clean image to pure noise (so information from the image isn't lost too early). The equation expresses that x_t is a noisy version of x_{t-1} , with the noise being drawn from a normal distribution.

2.1.2 Reverse Process

In the reverse process, the goal is to learn how to reverse the noise addition and recover the original image. The model learns to predict the noise ϵ that was added to the image at each step, based on the noisy input x_t . This predicted noise is then used to estimate the original clean image x_0 . Let $p_\theta(x_{t-1}|x_t)$ be the learned reverse transition model. This distribution tells us how to reverse the noise addition at each step. In practice, this distribution is modelled using a neural network $\epsilon_\theta(x_t, t)$, which predicts the noise added at each step. The reverse process can be represented as:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1} | \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

Where $\mu_\theta(x_t, t)$ is the mean of the reverse process predicted by the neural network, and $\Sigma_\theta(x_t, t)$ is the covariance.

The mean of the reverse process can be computed as:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{1 - \beta_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \beta_t}} \epsilon_\theta(x_t, t) \right)$$

Where $\epsilon_\theta(x_t, t)$ is the predicted noise at step t that was added during the forward process.

The model learns to predict the noise added during the forward process, and then it uses that prediction to remove the noise at each step. The reverse process can then be approximated by training on this distribution at each step, starting from the noisy image x_t and going backwards toward the clean image x_0 .

By learning how to reverse the forward diffusion process, the model can generate entirely new images by starting with random noise and iteratively denoising it into a coherent image based on a given input prompt or conditions.

2.1.3 Training

The training process aims to maximise the likelihood of generating a complex distribution that matches real data (such as images in the model's training dataset), from a simple Gaussian noise distribution.

The objective of training a diffusion model is to learn the parameters θ of the neural network $\epsilon_\theta(x_t, t)$ so that the reverse process accurately denoises the data. This is typically done by minimizing a loss function.

One of the ways this loss function is formulated is:

$$\mathcal{L}_\theta = \mathbb{E}_{q(x_0, \epsilon)} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

Which calculates the mean-squared error between the predicted and actual noise.

In some diffusion models, an alternative objective function is used, which involves minimizing the “Kullback-Leibler (KL) divergence” (a measure of how one probability distribution diverges from a second, expected distribution) between the forward and reverse processes.

$$\mathbb{E}_q \left[\sum_{t=1}^T D_{\text{KL}}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t)) \right]$$

This term penalizes the difference between the true reverse process (via the KL divergence) and the learned reverse process. By minimizing this divergence, the model improves its ability to reverse the forward diffusion process and generate realistic data.

2.1.4 Other Concepts

Guided Generation

Once trained, the diffusion model can generate new images. The standard method is unconditional generation, where the model generates samples from pure noise. However, we may want to guide this generation process - for instance, to generate specific types of images, such as from a given text prompt. To do this, we use the “Classifier Guidance” and “Classifier-Free Guidance” techniques:

Classifier Guidance Involves using a pre-trained classifier to steer the generation process toward specific classes (for instance, “cats”). The gradient of the log-probability of the target class with respect to the noisy image x_t is used to adjust the model’s predictions and guide the sample toward the desired class (Dhariwal and Nichol, 2021).

Classifier-Free Guidance This method removes the need for a separate classifier by training the denoising model with and without conditioning information (e.g., class labels or text prompts). At inference time, the model interpolates between the conditional and unconditional predictions using a *guidance scale* to control how strongly the generation is influenced by the conditioning signal (Ho and Salimans, 2022).

This guidance scale, denoted as s , controls the strength of the guidance. A higher value of s increases the influence of the conditioning, leading to outputs more closely aligned with the input prompt. However, excessively high values can lead to overfitting to the prompt, potentially reducing image quality. The guidance mechanism is mathematically expressed as:

$$\hat{\epsilon}(x_t, t) = (1 + s) \cdot \epsilon_\theta(x_t, t, \text{cond}) - s \cdot \epsilon_\theta(x_t, t, \text{uncond})$$

where $\epsilon_\theta(x_t, t, \text{cond})$ and $\epsilon_\theta(x_t, t, \text{uncond})$ are the model's predictions under the conditional and unconditional settings, respectively.

Latent Diffusion Models (LDMs)

Latent Diffusion Models (LDMs) are a class of generative models that extend the principles of diffusion models into a compressed latent space (Rombach et al., 2022). Traditional diffusion models operate directly in pixel space, progressively adding noise to an image and then learning to reverse this process to recover the original image. While effective, this approach is computationally intensive, especially for high-resolution images.

LDMs address this challenge by introducing an encoder-decoder architecture, where an encoder compresses the image into a lower-dimensional latent representation, and a decoder reconstructs the image from this latent space. The diffusion process can therefore be applied to this latent representation, significantly reducing the computational burden. Mathematically, the forward and reverse diffusion processes in the latent space are defined as:

$$q(z_t|z_{t-1}) = \mathcal{N}(z_t; \sqrt{1 - \beta_t}z_{t-1}, \beta_t I)$$

$$p_\theta(z_{t-1}|z_t) = \mathcal{N}(z_{t-1}; \mu_\theta(z_t, t), \Sigma_\theta(z_t, t))$$

where z_t represents the latent variable at time step t , and β_t is a small noise coefficient controlling the noise addition. The model learns to reverse the diffusion process in the latent space, allowing for efficient generation of high-resolution images.

Text-to-Image Generation with CLIP

Incorporating textual information into image generation involves conditioning the model on text prompts. One effective approach is to use a pre-trained text encoder, such as CLIP (Contrastive Language-Image Pre-training) (Radford et al., 2021), to obtain a fixed-length embedding of the input text. This embedding is then used to condition the generative model.

The conditioning is typically achieved through a cross-attention mechanism, where the text embedding influences the generation process at each step. This allows the model to generate images that are semantically aligned with the input text, facilitating controlled image synthesis based on natural language descriptions.

2.2 Stable Diffusion

Stable Diffusion (SD) is a open-source diffusion model developed by Stability AI, building upon the Latent Diffusion Models (LDMs) framework. SD incorporates recent advances in CLIP-based text conditioning, high-resolution latent diffusion modelling, and classifier-free guidance to generate high-quality images from natural language prompts.

The architecture and image generation pipeline of Stable Diffusion 2.0 consist of the following stages:

1. **Text Encoding:** The input prompt is tokenized and then encoded using a large-scale pre-trained OpenCLIP text encoder (ViT/H), producing a fixed-length embedding that captures semantic information from the text.
2. **Image Encoding:** Instead of operating directly in pixel space, the model applies the diffusion process in a compressed latent space. A variational autoencoder (VAE) encodes images into this space at a lower spatial resolution (e.g., 128×128 for 512×512 images), enabling computational efficiency without significant loss in visual fidelity.
3. **Guided Denoising with Classifier-Free Guidance:** During the reverse denoising process, the UNet denoiser is conditioned on the text embedding using cross-attention. Classifier-free guidance is used to blend the outputs of conditional and unconditional denoising passes:

$$\hat{\epsilon}_{\theta}(x_t, t) = (1 + s) \cdot \epsilon_{\theta}(x_t, t, \text{cond}) - s \cdot \epsilon_{\theta}(x_t, t, \text{uncond})$$

where s is a guidance scale hyperparameter (typically in the range of 5 to 10), controlling the trade-off between fidelity to the prompt and visual diversity.

4. **Image Decoding:** After iterative denoising, the final latent vector is decoded into the pixel space using again using the VAE decoder, yielding the final high-resolution image.

We will be considering standard Stable Diffusion-2 output as the baseline for the purposes of this project.

Chapter 3

Related Work

3.1 Textual Inversion



Figure 3.1: Textual Inversion being used to learn a concept (Sourced from: *An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion*)

Textual Inversion (Gal et al., 2022) is a method introduced to personalize pre-trained text-to-image models by associating new concepts with unique new tokens in their embedding space. This technique enables the generation of images that faithfully represent specific, user-defined concepts using minimal training data, typically 3–5 images. By embedding these concepts into the model’s latent space, Textual Inversion facilitates intuitive and flexible image generation through natural language prompts.

The core of Textual Inversion involves learning a new embedding vector that represents a user-defined concept. Starting with a pre-trained text-to-image model, the method introduces a new token, referred to as a "pseudo-word," into the model’s vocabulary. This token is associated with an embedding vector initialized randomly. The embedding vector is then optimized to minimize the difference between the generated images and the target concept, using a small set of images as references. The optimization process involves backpropagation through the model’s layers, adjusting the embedding vector to capture the unique features of

the concept. During training, the model is presented with textual prompts that include the new token. The objective is to generate images that align with both the textual description containing the new token (of the form "a photo of *token*") and the visual characteristics of the target concept 3.2.

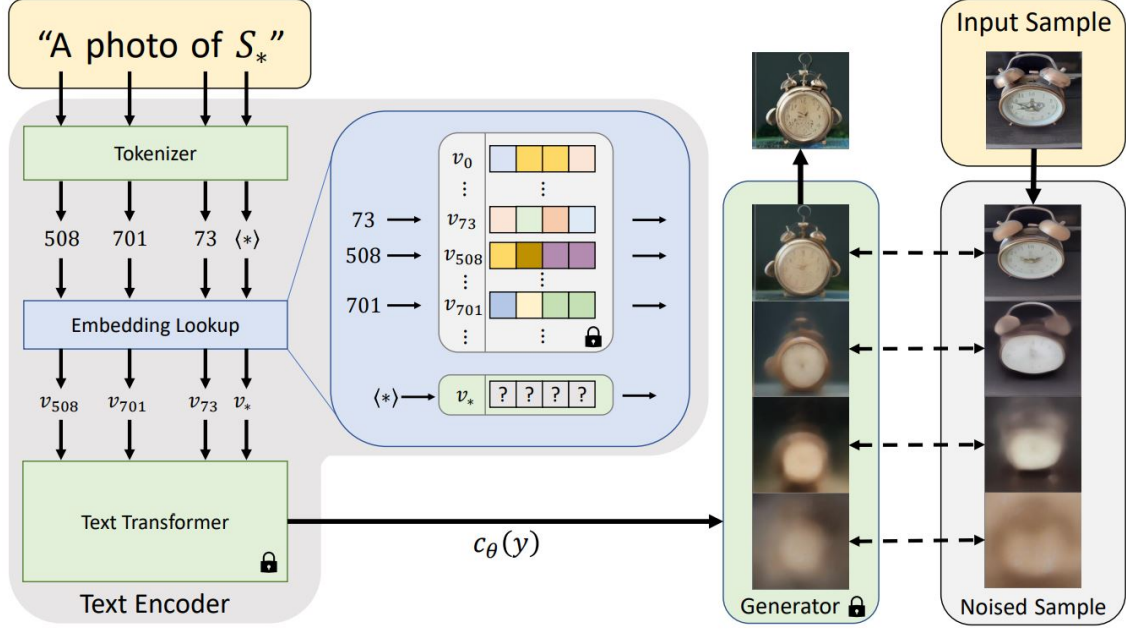


Figure 3.2: Textual Inversion Training - A description string containing the new pseudo-word is tokenized and then converted into continuous vector representations. The embedding vector $\mathbf{v}^*$ associated with the pseudo-word S^* , is what is to be learnt. (Sourced from: *An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion*)

The optimization objective is to minimize the difference between the predicted and true noise added during the forward diffusion process. This is formalized as:

$$\mathbf{v}_* = \arg \min_{\mathbf{v}} \mathbb{E}_{\mathbf{z} \sim \mathcal{E}(\mathbf{x}), \mathbf{y}, \epsilon \sim \mathcal{N}(0,1), t} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{z}_t, t, c_{\theta}(\mathbf{y}, \mathbf{v}))\|_2^2 \right]$$

Here, $\mathbf{v}$ is the embedding vector to be learned, $\mathcal{E}(\mathbf{x})$ denotes the encoder of the pre-trained model, $\mathbf{z}_t$ represents the latent code at a specific time-step t , $\epsilon_{\theta}(\mathbf{z}_t, t, c_{\theta}(\mathbf{y}, \mathbf{v}))$ is the denoising model's prediction, and $c_{\theta}(\mathbf{y}, \mathbf{v})$ is the conditioning vector derived from the prompt $\mathbf{y}$ and the learned embedding vector $\mathbf{v}$.

One of the significant advantages of Textual Inversion is its ability to integrate seamlessly into existing pre-trained models. By fine-tuning only the embedding vector associated with the new token, the method preserves the general knowledge encoded in the original model while incorporating the new concept. This selective fine-tuning is computationally efficient and enables rapid adaptation to new concepts without extensive retraining.

In this project, we will be employing this technique to similarly learn an embedding vector associated with a particular character in our comic, in order to be able to create visually consistent images of this character at different stages of our story.

3.2 Other Relevant Work

Apart from Textual Inversion, there are some other methods which have previously been employed to capture a concept or the likeness of a person consistently in generated images. A few of these are listed below:

- The paper *Zero-shot Generation of Coherent Storybook from Plain Text Story using Diffusion Models* (Jeong, Kwon and Ye, 2023) introduces a method to generate visual storybooks from plain text. Their approach employs a combination of textual inversion and semantic image editing techniques. This semantic image editing utilizes pre-trained detectors to identify and modify specific regions of an image, such as facial features, after the generation of the image ensuring that the person maintains a consistent appearance throughout the storybook.
- The paper *The Chosen One: Consistent Characters in Text-to-Image Diffusion Models* (Avrahami et al., 2024) presents a method for generating consistent character images from text prompts. It uses textual inversion to create a unique token representing the character and Low-Rank Adaptation (LoRA) to add to the model's attention layers. (LoRA fine-tunes pre-trained models by adding low-rank matrices (called adapters) to their weights, leaving the original weights unchanged, allowing for effective personalization without requiring extensive retraining. (Hu et al., 2021))
- The paper *DreamBooth: Fine-Tuning Text-to-Image Diffusion Models for Subject-Driven Generation* (Ruiz et al., 2023) introduces a method to ensure consistent subject generation by fine-tuning a pre-trained text-to-image diffusion model using a few images of a specific subject. This is done by adding a unique identifier token to the model's vocabulary and associating it with the subject's visual features - similarly to textual inversion. The model is then able to generate images that consistently depict the subject in various contexts, preserving key features such as appearance, pose, and lighting. DreamBooth differs from Textual Inversion in that it fine-tunes the entire model to embed the subject within its output domain, allowing for the generation of novel images of the subject while preserving its identity. In contrast, Textual Inversion learns the embedding for a new token in the embedding space of a frozen model, which can be less expressive and may struggle to capture intricate details of the target object.
- The paper *Training-Free Consistent Text-to-Image Generation* ((Tewel et al., 2024)) introduces "ConsiStory", a novel approach that ensures consistent subject generation across diverse prompts without requiring any model retraining. Instead of fine-tuning, ConsiStory leverages the internal activations of a pre-trained diffusion model during inference to maintain subject identity. Key to this method is the Subject-Driven Self-Attention (SDSA) mechanism, which modifies the standard self-attention layers in the U-Net decoder. SDSA allows each image in a batch to share information with others, guided by subject-specific masks. This sharing promotes consistent subject representation of common objects across different images.

I chose Textual Inversion over LoRA, DreamBooth, ConsiStory for my comic generation pipeline due to its efficiency, low resource requirements, and suitability for representing unique characters.

Textual Inversion requires very few images to learn a new concept, making it ideal for personalized character generation. In contrast, LoRA, typically necessitates a larger dataset

to achieve comparable results. DreamBooth offers high-quality personalization but demands significant computational resources and time for training. ConsiStory, doesn't require any additional training, but it may not perform well with uncommon or unique subjects, as it relies on broader training data to maintain consistency across generations.

Chapter 4

Methodology

Our proposed system, for generating comics with consistent characters from a single text prompt, is composed of four stages:

1. Image Prompt and Caption Generation via an LLM
2. Character Embedding Learning via Textual Inversion
3. Panel-wise Image Generation using the fine-tuned Stable Diffusion Pipeline
4. Artistic Stylization using Image-to-Image Style Transfer

4.1 Input Processing

Processing the singular text input is the first step in this comic generation pipeline. Here, we transform a simple text prompt into structured, multi-modal inputs for our textual inversion algorithm.

The process begins with a user-supplied comic prompt, such as:

```
COMIC_PROMPT = "Sarah visits the library"
```

or

```
COMIC_PROMPT = "Sarah, a tall woman with curly red hair wearing a  
yellow T-shirt and jeans, visits the library"
```

This comic prompt can contain both - the character description and the story description - or only a description of the story. In case there isn't a character description, a detailed visual description of the character is generated separately, using an LLM (For the purposes of this project, I have decided to make use of Google's Gemini LLM (gemini-1.5-pro)).

Firstly, we attempt to extract a character description clause from the comic prompt. If the original prompt does not include a visual description, the LLM generates one, such as for the description-less example above:

```
CHARACTER_PROMPT = "a tall woman with curly red hair wearing a  
yellow T-shirt and jeans"
```

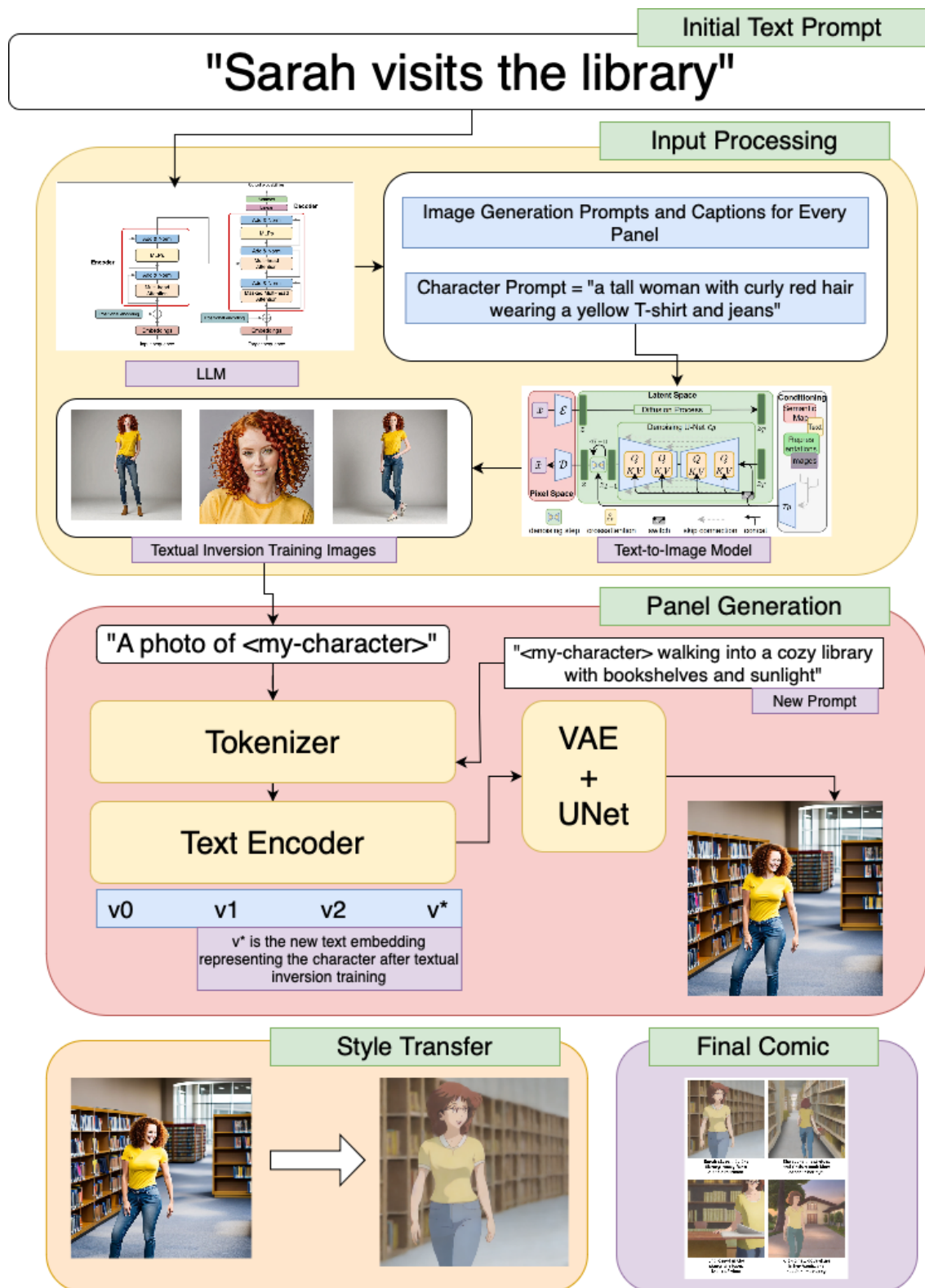


Figure 4.1: A diagram illustrating the complete pipeline

This way the user has the option to customize their character, only if they want to.

If the character's visual traits were extracted from the comic prompt, the LLM removes that portion from the prompt to isolate the narrative component.

This yields a concise story prompt - for example, both the above COMIC_PROMPTS will lead to the same story prompt.

```
STORY_PROMPT = "Sarah visits the library"
```

Next, we use the LLM again to expand the simplified story into a structured, multi-panel comic script - with image generations prompts, as well as panel captions. The image generation prompts must contain our chosen textual inversion pseudo-word (<my-character> in this case) to make sure the trained model can generate accurate images containing this character. An example output:

```
PROMPTS_AND_CAPTIONS = {
  "1": {
    "prompt": "<my-character> walking into a cozy library with
              bookshelves and sunlight",
    "caption": "Sarah steps into the library, ready for a quiet
              afternoon."
  },
  "2": {
    "prompt": "<my-character> reaching for a book on a shelf in the
              library",
    "caption": "She scans the shelves and finds a book that catches
              her eye."
  },
  "3": {
    "prompt": "<my-character> reading a book at a wooden table in
              the library",
    "caption": "Engrossed in the story, she loses track of time."
  },
  "4": {
    "prompt": "<my-character> walking out of the library holding a
              book, sunset sky",
    "caption": "With a new adventure in her hands, she heads home
              happy."
  }
}
```

At this point, we generate input images of the character, using the CHARACTER_PROMPT, in order to fine-tune the pre-trained diffusion model ("stabilityai/stable-diffusion-2" in our case). We will use these images to enable the model to learn how <my-character> looks.

Full-body and Portrait images of the character are generated from the CHARACTER_PROMPT using the pre-trained text-to-image model "lykon/dreamshaper-xl-turbo" to synthesize training images. I chose "lykon/dreamshaper-xl-turbo" for this task, because of its ability to produce high-quality photorealistic images of humans.

Following are the input images created using the above prompt 4.2.



Figure 4.2: Training images for textual inversion

4.2 Training Textual Inversion

To represent our unique character, we introduce a character token `<my-character>`, which is added to the model's tokenizer and trained to be associated with the character in the above images 4.2. The angular brackets ensure that the token is unlikely to clash with existing tokens in the model's vocabulary.

To understand the character's visual representation, we create a set of varied natural language prompts. Each prompt includes a placeholder (`{}`) that is replaced with `<my-character>` during training. This exposes the model to different ways the character might be referenced or described in text. The prompt templates are:

```
templates = [
    "a DSLR photo of {}",
    "a casual photo of {}",
    "a candid photo of {}",
    "a close-up photo of {}",
    "a cropped photo of {}",
    "a fashion photo of {}",
    "a good photo of {}",
    "a high-resolution photo of {}",
    "a photo of {}",
    "a photo of {} in a studio",
    "a photo of {} in natural light",
    "a photo of {} looking at the camera",
    "a photo of {} with cinematic lighting",
    "a portrait of {}",
    "a professional photo of {}",
    "a realistic photo of {}",
    "a rendering of {}",
    "a rendition of {}",
    "a studio-lit photo of {}",
    "a picture of {}",
    "an image of {}",
    "an old photo of {}",
    "the photo of {}"
]
```

Prompts are randomly sampled from the template list for each image instance, introducing variation into the training input. A custom dataset class that processes these image-prompt pairs during training, horizontally flips the training images with a probability of 0.5 for further augmentation of the training data.

Before training begins, the new token `<my-character>` is initialized using the embedding vector for the word "person" from the tokenizer's existing vocabulary. This gives the token a starting semantic meaning related to human figures.

To ensure the model does not forget prior learnt knowledge, all model parameters - including those of the UNet and VAE are frozen. Only the embedding vector for `<my-character>` is updated during training. This prevents overfitting or catastrophic forgetting, while focusing learning on associating the token with the character's visual features.

4.2.1 Standard Training

The images are first passed through stable diffusion's Variational Autoencoder (VAE). The VAE encodes each image $x \in \mathbb{R}^{H \times W \times C}$ into a lower-dimensional latent representation $z \in \mathbb{R}^{h \times w \times c}$.

Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$ is sampled and added to the latent representation z in a time-dependent manner. A random time-step t is drawn for each example in the batch. The noise scheduler (DDPM) then perturbs the latent as:

$$z_t = \sqrt{\bar{\alpha}_t} \cdot z + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$$

where $\bar{\alpha}_t$ is a predefined cumulative noise schedule (dependent on t) (Ho, Jain and Abbeel, 2020). This forms the "noisy latent" input for the denoising model.

In parallel, the textual prompts (of the form, "a photo of <my-character>") are tokenized and passed through stable diffusion's text encoder (CLIP). This yields a sequence of token embeddings.

The noisy latent z_t , the time-step t , and the text embedding vectors are then passed to a UNet-based denoising model. The UNet tries to predict the original noise ϵ that was added during the forward process:

$$\hat{\epsilon} = \text{UNet}(z_t, t, \text{text_embeddings})$$

This network outputs a tensor of the same shape as the latent, which represents the predicted noise. This predicted noise is then compared to the actual noise that was used during the forward diffusion step. The mean squared error between these two quantities serves as the loss function. This loss is back-propagated, and only the embedding vector associated with the character token is updated.

The central learning signal in textual inversion is derived from the discrepancy between the noise predicted by the model and the actual noise that was added to the latent representation of the image during the forward diffusion process. This is formalized as a Mean Squared Error (MSE) loss:

$$\text{loss} = \text{F.mse_loss}(\text{noise_pred}, \text{target}, \text{reduction}="none").\text{mean}([1, 2, 3]).\text{mean}()$$

Here, `noise_pred` denotes the output of the UNet model, which attempts to predict the noise added to the latent image at time-step t , and `target` refers to the true noise ϵ that was used to perturb the latent during the forward diffusion step.

4.2.2 Experimental Semantic Loss

In contrast to usual implementations of textual inversion, where the only inputs available are the training images, we also have access to the textual description of the character (which was used to generate the training images).

The standard textual inversion procedure (explained above) trains a new token embedding to minimize the error in denoising a latent representation of an image, conditioned on a prompt that includes this token. While effective, this process focuses solely on the reconstruction loss

in image space. It does not explicitly constrain the semantic meaning of the learned embedding in the textual space, which may result in the model capturing style, pose, or background elements from the training images while ignoring core visual traits of the character such as hair colour, clothing etc. Nothing inherently ensures this embedding retains semantic ties to the textual description of the character.

To address this we introduce a semantic loss term to our loss calculation. This auxiliary loss compares the embedding of a learned prompt (e.g., "a photo of <my_character>") to that of the fixed character description (e.g., "a tall woman with curly red hair wearing a yellow T-shirt and jeans") using cosine similarity. The intuition is to encourage the learned token to remain anchored in the semantic space defined by the reference description and penalize semantic drift from that description.

The character description ("a tall woman with curly red hair... ") is converted to a series of embeddings by the text encoder used with the diffusion model. This can help us calculate a fixed vector $\mathbf{E}_{\text{ref}} \in \mathbb{R}^d$, where d is the hidden dimension of the text encoder:

$$\mathbf{E}_{\text{ref}} = \text{Mean}(\text{TextEncoder}(\text{"a tall woman with curly red hair... "}))$$

During training, the encoder also produces contextual embeddings for each token in the template caption ("a photo of <my_character>"), which can be averaged to produce $\mathbf{E}_{\text{token}} \in \mathbb{R}^d$:

$$\mathbf{E}_{\text{token}} = \text{Mean}(\text{TextEncoder}(\text{"a photo of <my_character>"}))$$

A cosine similarity is computed between $\mathbf{E}_{\text{token}}$ and $\mathbf{E}_{\text{ref}}$, and a loss is defined as:

$$\mathcal{L}_{\text{semantic}} = 1 - \cos(\mathbf{E}_{\text{token}}, \mathbf{E}_{\text{ref}})$$

This loss is minimized when the angle between the two vectors is small, i.e., when they lie in similar directions in the embedding space, which implies semantic similarity.

The total loss now will be a weighted sum of the original image-space reconstruction loss $\mathcal{L}_{\text{recon}}$ and the semantic alignment loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \lambda \cdot \mathcal{L}_{\text{semantic}}$$

where λ is a tunable hyperparameter that governs the influence of the semantic constraint.

```
if TRAIN_WITH_TEXT_PROMPT:
    with torch.no_grad():
        # Get embedding for image caption e.x. "A photo of
        # {CH_TOKEN}"
        char_input_ids = batch["input_ids"]
        char_embedding = text_encoder(char_input_ids)[0]
        char_embedding_mean = char_embedding.mean(dim=1) # shape:
        (batch_size, hidden_dim)
```

```

# Get embedding for the fixed character description
target_ids = tokenizer(Character_PROMPT,
                        return_tensors="pt").input_ids.to(accelerator.device)
target_embedding = text_encoder(target_ids)[0]
target_embedding_mean = target_embedding.mean(dim=1) #
                        shape: (1, hidden_dim)

# Compute cosine similarity between each char_embedding and the
  fixed description
embedding_loss = 1 - F.cosine_similarity(
    char_embedding_mean,
    target_embedding_mean.expand_as(char_embedding_mean)
).mean()

loss = loss + (0.2 * embedding_loss)

```

This addition tries to address a key limitation in standard textual inversion: the lack of explicit semantic grounding of the added token in the text space. By ensuring that the learned token embedding remains close to the original description, the model is better encouraged to retain identity-specific attributes, such as hair colour, clothing, and physical traits, and reduce overfitting to low-level visual features like camera angle or background elements, which may dominate the loss signal in standard textual inversion. We will further compare this approach with standard textual inversion in the Results section.

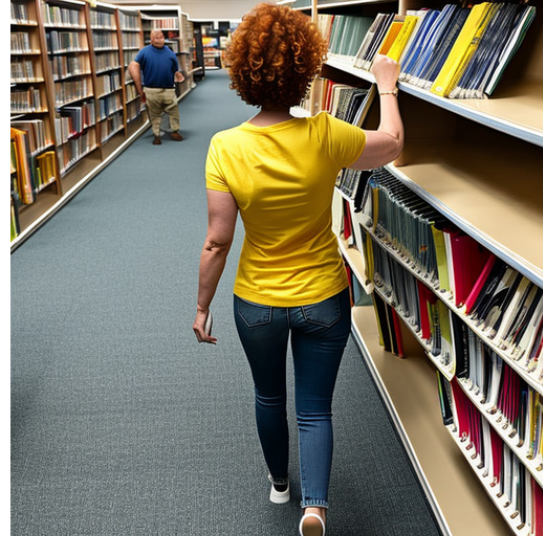
4.3 Panel-wise Image Generation

After training, the Stable Diffusion pipeline (with the newly learnt text embedding) is used to generate the individual panels of the comic based on the prompts generated beforehand.

These are the results for our "Sarah visits the library" example (using simple textual inversion without using semantic loss) 4.3.



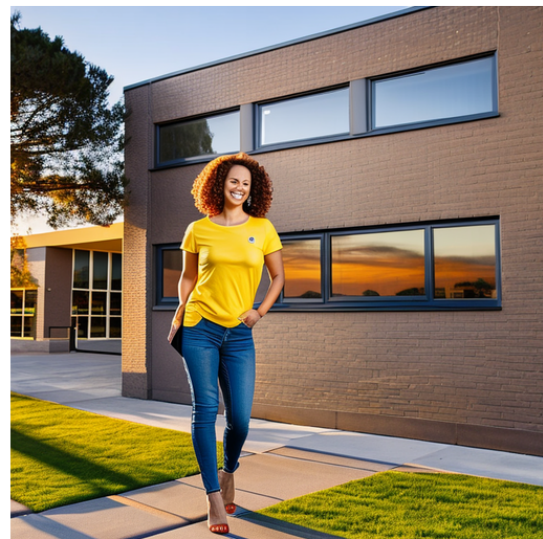
Sarah steps into the library, ready for a quiet afternoon.



She scans the shelves and finds a book that catches her eye.



Engrossed in the story, she loses track of time.



With a new adventure in her hands, she heads home happy.

Figure 4.3: Comic panels generated by the diffusion model after standard textual inversion.

4.4 Style Transfer

To complete this comic-creation process the last step is to take our generated panel images and convert them to images in a hand-drawn art-style, more likely to be seen in a traditional comic/manga - using a dataset of stills from animated Studio Ghibli movies.

Why Style Transfer?

We will be using style transfer from photorealistic images to Ghibli-style images, instead of fine-tuning a diffusion model to directly generate stylized images - because of the distinct colour characteristics of the images in the Ghibli dataset - specifically, their pastel palette and limited colour diversity. An analysis of the Ghibli Dataset reveals that Ghibli art typically uses a very limited set of colours - mostly soft pastels 4.6, with reduced saturation and contrast compared to the real-world photographs used to train most diffusion models. The use of nature-focused settings and backgrounds has also skewed the image dataset to have a much higher concentration of pixels with a high green channel value 4.54.7. Fine-tuning a generative model on such images can lead to catastrophic forgetting of the broader, more diverse colour distributions it originally learned. This makes the model less flexible and may actually reduce its generalization capacity outside that narrow colour regime. In contrast, employing style transfer will allow us to preserve the content and structural diversity from high-diversity datasets while applying the colour and texture features of Ghibli images, producing output that retains its diversity but adheres to the Ghibli aesthetic.

To this end, we fine-tune another diffusion model, on a dataset of hand-drawn artwork using LoRA Hu et al. (2021).

4.4.1 Introduction to LoRA

Low-Rank Adaptation (LoRA) is a method used to adapt pre-trained models to new tasks or domains without retraining the entire model. The key idea is to decompose the weight updates into low-rank matrices, which dramatically reduces the number of parameters that need to be learned. This approach is particularly beneficial when adapting large models, like those used in image generation, as it reduces computational overhead while still allowing for significant fine-tuning.

Given a pre-trained model f_θ , where θ denotes the model's parameters, LoRA introduces a set of low-rank updates $\Delta\theta$ to adapt the model. Instead of directly updating the entire parameter set θ , LoRA introduces two low-rank matrices A and B such that:

$$\Delta\theta = A \cdot B$$

Here, A and B are matrices of much smaller dimensions compared to the original weight matrices. This decomposition ensures that only a fraction of the parameters need to be learned during adaptation, which can be a significant computational advantage when dealing with large-scale models.

Mathematically, LoRA adapts the layers of the model using the decomposition:

$$\theta' = \theta + \Delta\theta = \theta + A \cdot B$$

where θ' represents the adapted weights.

Since LoRA fine-tuning adjusts the effect of a model's weights, it is able to capture subtle, global features such as colour schemes, brushwork, and lighting effects in an image. This capability makes LoRA particularly effective for learning and replicating intricate, abstract concepts like artistic styles. Therefore, it is a suitable choice for capturing comic-like art styles.

4.4.2 Dataset

To train the diffusion model for style transfer, we will be fine-tuning it using images from this dataset of Studio Ghibli style artwork which was used to train AnimeGANv2.

The dataset contains approximately 1800 unlabelled images from various movies from Studio Ghibli, a renowned Japanese animation studio^{4.4}.

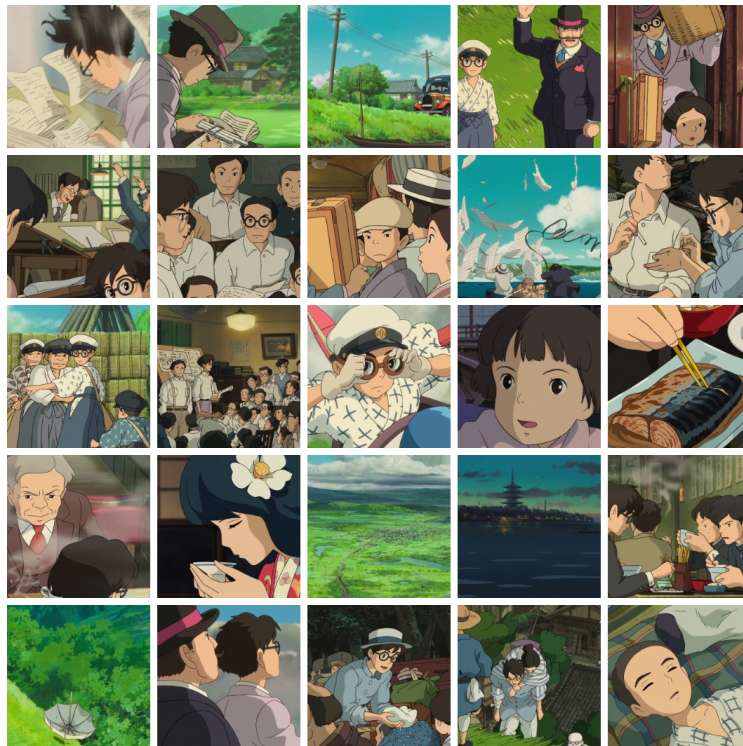


Figure 4.4: A selection of images from the Studio Ghibli Dataset

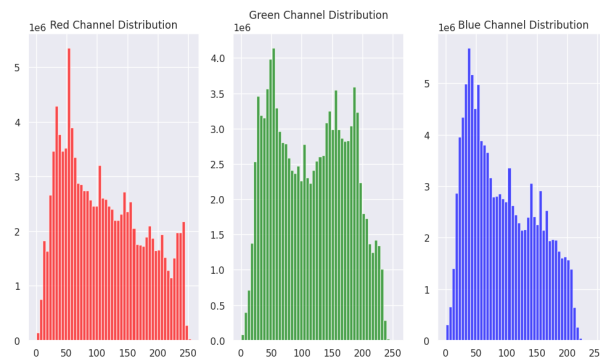


Figure 4.5: Distribution of RGB values across all pixels in the Studio Ghibli Dataset

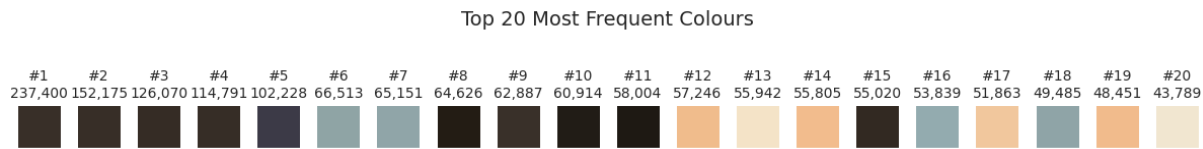


Figure 4.6: The top 20 most commonly used colours across the Studio Ghibli Dataset



Figure 4.7: Mean Colour of the Studio Ghibli Dataset

To train this LoRA, we will first need to generate text captions for the Studio Ghibli-style images in our dataset. We accomplish this by using the BLIP (Bootstrapping Language-Image Pre-training) model (Li et al., 2022), a vision-language pre-trained model capable of generating natural language captions based on visual inputs. BLIP employs a multimodal mixture of encoder-decoder (MED) architecture, which includes a visual transformer for image encoding and a BERT-based text encoder, enhanced with cross-attention layers to integrate visual information. The model is pre-trained using three objectives: Image-Text Contrastive Loss (ITC), Image-Text Matching (ITM), and Language Modeling (LM), enabling it to perform tasks like image captioning, visual question answering, and image-text retrieval.

When generating these captions we make sure to exclude specific words such as "cartoon", "drawing" or "anime", which might describe this art style, from the captions, because we want the model to implicitly learn to associate these features in the training dataset with the Ghibli art style.

The captions are also augmented by adding style-specific prefixes, such as "A Ghibli style image showing " so the model learns to associate prompts like "Ghibli style" with images of a particular art style.

Finally, LoRA is applied by injecting low-rank adapters into the core layers of the Stable Diffusion model.

The LoRA parameters, such as rank r , alpha α , and dropout p , are fine-tuned for the task. These parameters control the capacity and efficiency of the low-rank adaptation. The rank r determines the size of the matrices A and B , while α controls the scaling of the low-rank update, and the dropout parameter p introduces regularization to prevent overfitting.

During training, the captions are tokenized and encoded into text embeddings, which are then used in conjunction with the noisy latents of the images. The network learns to predict the noise added to these latents, thereby learning the stylistic features of the Ghibli art form. The loss function is based on mean squared error (MSE), measuring the difference between the predicted and actual noise values.

Once the model has been fine-tuned, it can be used for both text-to-image generation and image-to-image style transfer. For image-to-image style transfer, an existing image can be transformed into the Ghibli style by providing the model with an input image and a prompt containing words to trigger generation in the trained art style like "a Ghibli style image". The strength of the style transfer can be adjusted using a "strength" parameter, which controls

how closely the output image adheres to the original style.

This is what the final comic looks like after the panels were converted to the Ghibli style and assembled in a comic strip 4.8.



Sarah steps into the library, ready for a quiet afternoon.



She scans the shelves and finds a book that catches her eye.



Engrossed in the story, she loses track of time.



With a new adventure in her hands, she heads home happy.

Figure 4.8: The final comic produced by our pipeline for the prompt "Sarah visits the library".

Chapter 5

Results

5.1 Output Evaluation

The Figure 5.1 displays iterations of the comic generated for our prompt "Sarah visits the library".

At first glance, we can see that our textual inversion algorithm has been reasonably successful at producing comics containing characters with certain consistent features like - hair and clothing.

In the top left image (standard textual inversion before style transfer) we can see that there are aspects of the character that have changed from the training images 4.2 - for instance the character's hair is curly but the red colour from the training image has change into a brown.

This red hair colour is better preserved in the top-right panel (textual inversion with semantic loss), possibly because this time the character text embedding was conditioned on the character prompt containing the words "red hair". However, the bottom-left panel of this comic does display yellow streaks in the character's red hair which may also be a result of the character description containing the word "yellow" in it.

Our approach of taking the mean of all embeddings of the character prompt "a tall woman with curly red hair..." and calculating it's cosine similarity with the mean of another embedding containing the character token, means that the model may have trouble associating the colour red, and not yellow, with the hair. This means that the model struggles with handling a prompt where it must bind two separate attributes (colours) to two separate objects (hair, clothing).

The Ghibli style transfer in both cases has remained fairly accurate in capturing the content of the originally generated panel. A major problem here has been the fact that all the character's images have artefacts of what look like a pair of glasses of her eyes. This is most likely because the Ghibli dataset that we used contain a significant number of images of "Jiro Horikoshi" from the film "The Wind Rises" - a character with glasses, which is why the model seems to have learnt the presence of glasses on characters' eyes as a feature of the art style.



Figure 5.1: Top Left: Standard Textual Inversion before style transfer. Bottom Left: Standard Textual Inversion after style transfer. Top Right: Textual Inversion employing semantic loss before style transfer. Bottom Right: Textual Inversion employing semantic loss after style transfer.



Figure 5.2: Comic A: Comic generated by Stable Diffusion 2-1 for the prompt "A comic strip for "Sarah visits the library". Retrieved from HuggingFace.



Figure 5.3: Comic B: Comic generated by an online image generator for the prompt "A comic strip for "Sarah visits the library". Retrieved from pixlr



Figure 5.4: Comic C: Our pipeline with standard textual inversion



Figure 5.5: Comic D: Our pipeline with textual inversion employing semantic loss

5.2 User Study

To better evaluate our comic generation pipeline, a User Study was conducted - asking participants to rate four different AI-generated comics, created with the same text prompt - A comic strip for - "Sarah visits the library".

Each version was rated on a likert scale with values ranging from 1 (Poor) to 10 (Excellent) on:

- Character consistency
- Story coherence
- Adherence to prompt
- Image quality

The participants could also optionally give an explanation for their selections for us to get better insight into what motivated their choices.

Displayed below is the feedback collected from the 59 participants that completed this survey.

Table 5.1: Summary Statistics

Comic	Character Consistency			Story Coherence			Adherence to Prompt			Image Quality		
	Mean	Median	Std	Mean	Median	Std	Mean	Median	Std	Mean	Median	Std
A	4.25	4.0	2.25	3.98	3.0	2.84	5.71	6.0	2.58	4.73	5.0	2.54
B	6.07	6.0	2.48	5.25	5.0	2.88	6.81	7.0	2.40	7.32	8.0	2.17
C	8.22	8.0	1.76	8.64	9.0	1.42	8.58	9.0	1.58	8.20	9.0	1.69
D	8.22	9.0	1.75	8.85	9.0	1.17	8.88	9.0	1.22	8.34	9.0	1.52

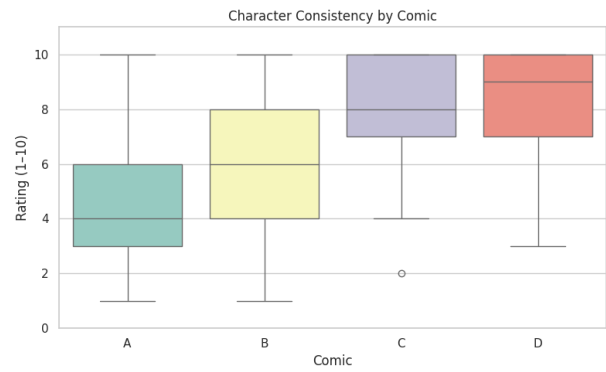


Figure 5.6: Boxplot of the four comics evaluated on "Character Consistency".

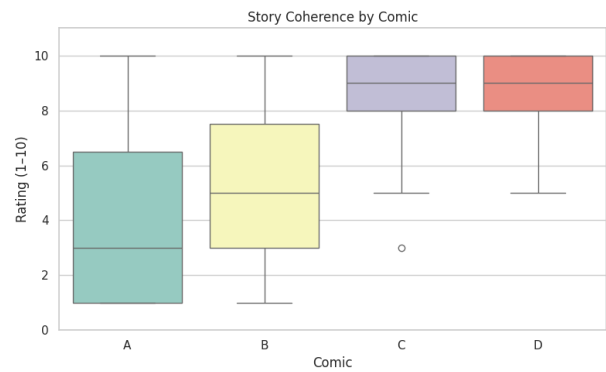


Figure 5.7: Boxplot of the four comics evaluated on "Story Coherence"

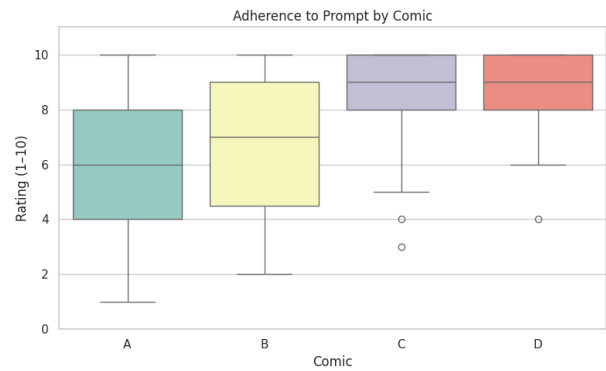


Figure 5.8: Boxplot of the four comics evaluated on "Adherence to Prompt"

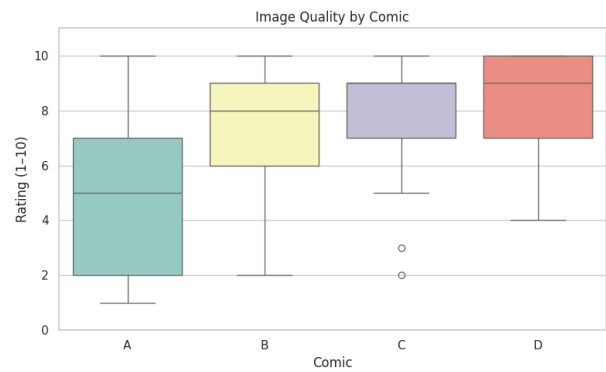


Figure 5.9: Boxplot of the four comics evaluated on "Image Quality"

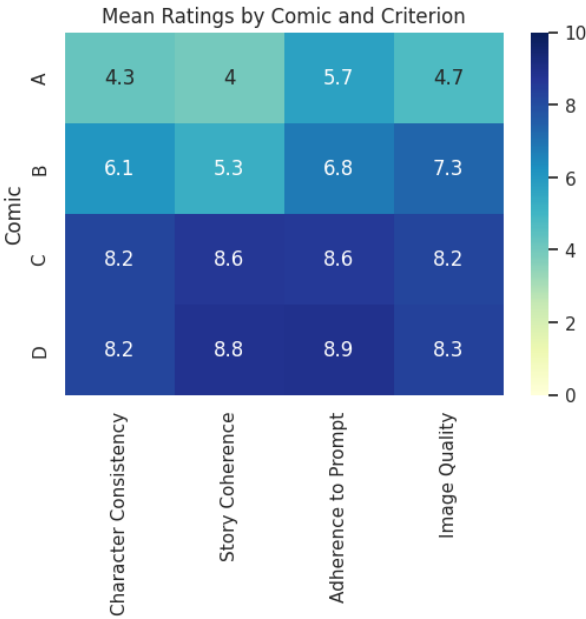


Figure 5.10: Heatmap of Comics A, B, C and D evaluated on the above four metrics. The darker colours (depicting higher values) are concentrated at the bottom of this figure, showing that Comics C and D (produced by our pipeline) vastly outperform diffusion models that generate the whole comic altogether.

Table 5.2: Percentage of Ratings Below 4

Comic	Character Consistency	Story Coherence	Adherence to Prompt	Image Quality
A	45.76%	52.54%	22.03%	35.59%
B	20.34%	35.59%	11.86%	10.17%
C	1.69%	1.69%	1.69%	3.39%
D	1.69%	0.00%	0.00%	0.00%

Comic	Mean of All Criteria
A	4.67
B	6.36
C	8.41
D	8.57

Table 5.3: Mean of all four criteria for each comic

The results from the user ratings for four AI-generated comics - Comics A, B, C, and D highlight significant differences in quality, with Comics C and D (produced by our pipeline) consistently outperforming Comics A and B across multiple criteria.

1. Overview of Quantitative Results

The results show a clear progression in quality from Comic A to Comic D. Comics C and D consistently outperformed A and B across all metrics, especially in coherence and character consistency. Let’s explore each comic in depth.

Metric	Comic A	Comic B	Comic C	Comic D
Character Consistency (Mean)	4.25	6.07	8.22	8.22
Story Coherence (Mean)	3.98	5.25	8.64	8.85
Adherence to Prompt (Mean)	5.71	6.81	8.58	8.88
Image Quality (Mean)	4.73	7.32	8.20	8.34

Table 5.4: Average ratings by metric for each comic.

2. Comic A

Quantitative Summary

- Lowest scores across all metrics.
- 45.76% rated Character Consistency below 4.
- 52.54% rated Story Coherence below 4.
- Image Quality and Adherence to Prompt also underwhelming.

Qualitative Feedback

Participants found Comic A confusing and disjointed. Key issues included:

- **Inconsistent characters:** Sarah appeared as multiple, differing individuals.
- **Poor panel alignment:** Panels were unaligned or cut off, making the comic hard to follow.
- **Weak narrative:** There was no discernible story or dialogue.
- **Limited adherence to the prompt:** While set in a library, the setting was often unclear.

While a few praised the comic’s visual style or colours, most agreed it felt incomplete and lacked clarity.

3. Comic B

Quantitative Summary

- Improved scores across all categories.
- 20.34% rated Character Consistency below 4.
- Image Quality mean improved to 7.32.

Qualitative Feedback

While Comic B was better received than A, it had lingering issues:

- **Unreadable text:** Gibberish dialogue made the story hard to understand.
- **Visual inconsistencies:** Sarah’s hair and clothing changed across panels.

- **Incomplete figures:** Some characters were only partially rendered.
- **Improved environment:** Library setting was clearer than before.

Some participants found it engaging, but incoherent dialogue and inconsistencies prevented it from being truly effective.

4. Comic C

Quantitative Summary

- High scores across all metrics.
- Only 1.69% of participants rated it's Character Consistency and Story Coherence below 4.

Qualitative Feedback

Comic C was praised as the first cohesive entry:

- **Strong coherence:** The narrative was clear and logical.
- **Consistent character:** Sarah was recognizable throughout, with minor distortions.
- **Readable dialogue:** Text was clear and contributed to story flow.
- **Faithful to prompt:** The comic clearly fulfilled the prompt scenario.

Minor critiques included odd facial features or slight outfit inconsistencies. Nevertheless, it marked a significant improvement in storytelling.

5. Comic D

Quantitative Summary

- Top performer in almost every metric.
- Story Coherence (8.85) and Adherence to Prompt (8.88) were highest overall.
- Character Consistency on par with Comic C (8.22).

Qualitative Feedback

Comic D was seen as the most refined of the set:

- **Coherent story:** Participants described it as easy to follow and logical.
- **Strong visual quality:** Blurriness was noted but didn't hinder comprehension.
- **Prompt adherence:** Universally agreed to be accurate to the task.
- **Character fidelity:** Some outfit and hairstyle changes were observed but largely forgiven.

A few mentioned the last panel felt slightly different in tone or rendering quality, but overall Comic D was considered the most complete of the four.

6. Comparative Insights

Metric Analysis

- **Character Consistency:** A major issue early on, improved significantly in C and D.
- **Story Coherence:** The most important metric to users; improved with readable text and logical flow.
- **Adherence to Prompt:** Generally satisfied, but clearer execution helped improve perception.
- **Image Quality:** Blurry, unfinished images were a problem initially, resolved in later entries.

This user study demonstrates a clear progression in quality and coherence from Comics A and B, to those produced by our fine-tuned pipelines (Comics C and D). The significant improvement in ratings - particularly in character consistency and story coherence - validates our approach of modular comic generation using textual inversion.

Chapter 6

Conclusions

6.1 Limitations and Future Work

Despite some of the successes of our pipeline, there are several limitations and opportunities for future work. Most notably, our current approach supports only a single character. Extending the method to handle multiple characters - using multiple token embeddings and a richer prompt-token mapping system - represents a promising direction for future research in multi-character comic generation.

Moreover, for the purposes of this project, the comic layout used was a static and pre-defined grid of images. This is far from actual comics which utilise diverse layouts with differently sized panels on their pages to keep readers engaged. Integrating recent advances in diverse layout generation, such as those discussed in *Manga Generation via Layout-controllable Diffusion* (Chen et al., 2024), would enable us to dynamic panel compositions, based on the content of the panels - resulting in a more entertaining experience for the readers.

Another important aspect lacking from our model is the presence of dialogue and speech bubbles, an extremely common feature of comic strips. Future work can incorporate ideas like those proposed in *Automatic Comic Generation with Stylistic Multi-page Layouts and Emotion-driven Text Balloon Generation* Yang et al. (2021), which use multimodal signals (such as, subtitle text and audio) to generate emotionally expressive text balloons that vary in shape and placement based on speaker identity and affect - when generating comics from video. The systems also address speaker detection, ensuring balloons are properly associated with the correct character - a significant feature in multi-character scenes.

For a more sophisticated comic dialogue generation system, integration with *MPDIALOG* (Agrawal et al., 2023), a large-scale multimodal model for comic dialogue generation, which takes into account the persona of the characters, in addition to the visual elements of a comic strip offers exciting potential. *MPDIALOG* allows for persona-conditioned conversational modelling, ensuring that character speech remains consistent with predefined personality traits. This would significantly enhance the coherence and believability of any AI-generated narratives by linking visual, textual, and behavioural signals together to create a three-dimensional character.

6.2 Final Remarks

This dissertation presented a novel pipeline for generating character-consistent, stylistic and coherent comic strips by leveraging diffusion models, along with the fine-tuning processes of - Textual Inversion and Low-Rank Adaptation (LoRA). To enhance our textual inversion token's understanding of the character we introduced a Semantic Loss term, the efficacy of which was evaluated in the a User Study. The results of the Study, strongly favoured our proposed methods, particularly the use of semantic loss as seen in Comic D 5.5. Improvements were noted in the metrics of character consistency, story coherence, adherence to prompt, and overall image quality. These findings validate the utility of our modular approach to comic generation and of the semantic grounding of character representations in the embedding space for high-fidelity comic generation.

In conclusion, while challenges remain - such as scaling to multiple characters, integrating speech, and enhancing narrative layouts - this pipeline provides a strong foundation for future research in AI-assisted storytelling. With the continued advancement of multimodal learning and generative models, we anticipate that the future of comic creation will be increasingly collaborative, with AI tools enabling artists and writers to bring complex stories to life more efficiently than ever before.

Bibliography

- Agrawal, H., Mishra, A., Gupta, M. and Mausam, 2023. Multimodal persona based generation of comic dialogs [Online]. In: A. Rogers, J. Boyd-Graber and N. Okazaki, eds. *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*. Toronto, Canada: Association for Computational Linguistics, pp.14150–14164. Available from: <https://doi.org/10.18653/v1/2023.acl-long.791>.
- Avrahami, O., Hertz, A., Vinker, Y., Arar, M., Fruchter, S., Fried, O., Cohen-Or, D. and Lischinski, D., 2024. The chosen one: Consistent characters in text-to-image diffusion models [Online]. *Special interest group on computer graphics and interactive techniques conference conference papers '24*. ACM, SIGGRAPH '24, p.1–12. Available from: <https://doi.org/10.1145/3641519.3657430>.
- Bradford, W. ca., 2011. Reaching the visual learner: Teaching property through art. *Social science research network* [Online]. Available from: <https://api.semanticscholar.org/CorpusID:142897142>.
- Chen, S., Li, D., Bao, Z., Zhou, Y., Tan, L., Zhong, Y. and Zhao, Z., 2024. Manga generation via layout-controllable diffusion [Online]. 2412.19303, Available from: <https://arxiv.org/abs/2412.19303>.
- Dhariwal, P. and Nichol, A., 2021. Diffusion models beat gans on image synthesis [Online]. 2105.05233, Available from: <https://arxiv.org/abs/2105.05233>.
- Duncan, R. and Smith, M.J., 2009. *The power of comics: History, form and culture*. A&C Black.
- Gal, R., Alaluf, Y., Atzmon, Y., Patashnik, O., Bermano, A.H., Chechik, G. and Cohen-Or, D., 2022. An image is worth one word: Personalizing text-to-image generation using textual inversion [Online]. 2208.01618, Available from: <https://arxiv.org/abs/2208.01618>.
- Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y., 2014. Generative adversarial networks [Online]. 1406.2661, Available from: <https://arxiv.org/abs/1406.2661>.
- Ho, J., Jain, A. and Abbeel, P., 2020. Denoising diffusion probabilistic models [Online]. 2006.11239, Available from: <https://arxiv.org/abs/2006.11239>.
- Ho, J. and Salimans, T., 2022. Classifier-free diffusion guidance [Online]. 2207.12598, Available from: <https://arxiv.org/abs/2207.12598>.
- Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W., 2021. Lora: Low-rank adaptation of large language models [Online]. 2106.09685, Available from: <https://arxiv.org/abs/2106.09685>.

- Jeong, H., Kwon, G. and Ye, J.C., 2023. Zero-shot generation of coherent storybook from plain text story using diffusion models [Online]. 2302.03900, Available from: <https://arxiv.org/abs/2302.03900>.
- Kumar, J.T., Babu, H. and Srinivasan, N., 2024. Comic generation using ai—a review. In: M.L. Owoc, F.E. Varghese Sicily, K. Rajaram and P. Balasundaram, eds. *Computational intelligence in data science*. Cham: Springer Nature Switzerland, pp.156–166.
- Li, J., Li, D., Xiong, C. and Hoi, S., 2022. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation [Online]. In: K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu and S. Sabato, eds. *Proceedings of the 39th international conference on machine learning*. PMLR, *Proceedings of Machine Learning Research*, vol. 162, pp.12888–12900. Available from: <https://proceedings.mlr.press/v162/li22n.html>.
- McCloud, S. and Martin, M., 1993. *Understanding comics: The invisible art*, vol. 106. Kitchen sink press Northampton, MA.
- Oktaviana, A.A., Joannes-Boyau, R., Hakim, B., Burhan, B., Sardi, R., Adhityatama, S., Hamrullah, Sumantri, I., Tang, M., Lebe, R. et al., 2024. Narrative cave art in indonesia by 51,200 years ago. *Nature*, 631(8022), pp.814–818.
- Pange, J., 2023. The use of comics as a teaching and learning tool. In: D. Guralnick, M.E. Auer and A. Poce, eds. *Innovative approaches to technology-enhanced learning for the workplace and higher education*. Cham: Springer International Publishing, pp.281–289.
- Proven-Bessel, B., Zhao, Z. and Chen, L., 2021. Comicgan: Text-to-comic generative adversarial network [Online]. 2109.09120, Available from: <https://arxiv.org/abs/2109.09120>.
- Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G. and Sutskever, I., 2021. Learning transferable visual models from natural language supervision [Online]. 2103.00020, Available from: <https://arxiv.org/abs/2103.00020>.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P. and Ommer, B., 2022. High-resolution image synthesis with latent diffusion models [Online]. 2112.10752, Available from: <https://arxiv.org/abs/2112.10752>.
- Ruiz, N., Li, Y., Jampani, V., Pritch, Y., Rubinstein, M. and Aberman, K., 2023. Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation [Online]. 2208.12242, Available from: <https://arxiv.org/abs/2208.12242>.
- Tewel, Y., Kaduri, O., Gal, R., Kasten, Y., Wolf, L., Chechik, G. and Atzmon, Y., 2024. Training-free consistent text-to-image generation [Online]. [Online], 43(4). Available from: <https://doi.org/10.1145/3658157>.
- Tseng, C.I., Laubrock, J. and Pflaeging, J., 2018. *Character developments in comics and graphic novels: Digital, multimodal, and cognitive methods* [Online], pp.154–175. Available from: <https://doi.org/10.4324/9781315185354-8>.
- Vivoli, E., Souibgui, M.A., Barsky, A., Llabrés, A., Bertini, M. and Karatzas, D., 2025. One missing piece in vision and language: A survey on comics understanding [Online]. 2409.09502, Available from: <https://arxiv.org/abs/2409.09502>.

- Yang, X., Ma, Z., Yu, L., Cao, Y., Yin, B., Wei, X., Zhang, Q. and Lau, R.W.H., 2021. Automatic comic generation with stylistic multi-page layouts and emotion-driven text balloon generation [Online]. 2101.11111, Available from: <https://arxiv.org/abs/2101.11111>.

Appendix A

Code

Below is some of the Python code developed as a part of this project (originally in Jupyter Notebooks). All the code files relevant to this project will be available on my GitHub

A.1 File: ConsistentCharacterComics.py

```
# %% [markdown]
# # Consistent Character Comics – Individual Project (Shehij Raina)

# %% [markdown]
# ### Imports and Initial Setup

# %%
# Manage Hex Cloud GPU Usage...
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "0"

# %%
# Import Libraries

import accelerate
import google.generativeai as genai
import json
import math
import numpy as np
import PIL
import random
import textwrap
import torch
import torch.nn.functional as F

from accelerate import Accelerator
from diffusers import AutoPipelineForImage2Image, AutoPipelineForText2Image, AutoencoderKL, DDPMScheduler, DPMSolverMultistepScheduler, StableDiffusionPipeline, UNet2DConditionModel
from huggingface_hub import login
from PIL import Image, ImageDraw, ImageFont
from torch.optim import AdamW
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from transformers import CLIPTextModel, CLIPTokenizer
from tqdm.auto import tqdm

# %% [markdown]
# ### Helper Functions

# %%
# Helps visualise a set of images
def display_images_in_grid(images, num_row, num_col):
    if len(images) != num_row * num_col:
```

```
        raise ValueError("Recheck dimensions for the image grid!")

    width, height = images[0].size # dimensions of an image

    grid = Image.new('RGB', size=(num_col * width, num_row * height))
    grid_width, grid_height = grid.size # dimensions of complete grid

    for i, image in enumerate(images):
        grid.paste(image, box=(i % num_col * width, i // num_col * height))

    display(grid)

# %% [markdown]
# ### Comic Prompt Processing

# %%
COMIC_PROMPT = "Sarah visits the library"
NUM_PANELS = 4

# %%
# This section processes a textual prompt to generate a structured comic story...

# Here we will extract the description of our character
# as well as the story to be told through the comic
# using a pretrained LLM such as Google Gemini

# Configure and create the gemini model
genai.configure(api_key="YOUR_API_KEY") # Replace with an actual key
model = genai.GenerativeModel('gemini-1.5-pro')

# Extract character description to isolate their visual identity...
EXTRACT_CHARACTER_QUERY = f'Extract the character description clause from "{COMIC_PROMPT}". No added words or punctuation. If there is no description generate a simple physical description of the character for image generation.'
CHARACTER_PROMPT = model.generate_content(EXTRACT_CHARACTER_QUERY).text.strip().lower().replace("character description found.\n\n", "").strip()
print(f'CHARACTER_PROMPT: {CHARACTER_PROMPT}')
```

```

# Extract narrative story and remove character description...
EXTRACT_STORY_QUERY = f'Remove the character description
    "{CHARACTER_PROMPT}" from the prompt "{COMIC_PROMPT}" (if
    there is one) and return a simple story prompt.'
STORY_PROPMT =
    model.generate_content(EXTRACT_STORY_QUERY).text.strip()
print(f'STORY_PROPMT: {STORY_PROPMT}')

# Generate comic-strip captions...
GENERATE_CAPTIONS_QUERY = f'Generate a {NUM_PANELS} panel comic
    story for the prompt "{STORY_PROPMT}". Provide short simple
    image generation prompts along with captions for each panel.
    Do not describe the character in the image generation prompts
    and refer to them as "<my-character>". Please return your
    response as valid json, where the panel number is a key and
    the value is another dictionary with two keys: "prompt" and
    "caption".'
response =
    model.generate_content(GENERATE_CAPTIONS_QUERY).text.strip()

# Process returned string into a python dictionary
json_text = response.replace("'", '"')
    json_text = json_text.replace("\n", "")
PROMPTS_AND_CAPTIONS = json.loads(json_text)

print(f'PROMPTS_AND_CAPTIONS: {PROMPTS_AND_CAPTIONS}')

# %%
# For the example within the dissertation the generated results
# were...

# CHARACTER_PROMPT = "a tall woman with curly red hair wearing
# a yellow T-shirt and jeans"
# STORY_PROPMT = "Sarah visits the library"
# PROMPTS_AND_CAPTIONS = {
#     "1": {
#         "prompt": "<my-character> walking into a cozy library with
# bookshelves and sunlight",
#         "caption": "Sarah steps into the library, ready for a quiet
# afternoon."
#     },
#     "2": {

```

```

#         "prompt": "<my-character> reaching for a book on a shelf in
# the library",
#         "caption": "She scans the shelves and finds a book that
# catches her eye."
#     },
#     "3": {
#         "prompt": "<my-character> reading a book at a wooden table
# in the library",
#         "caption": "Engrossed in the story, she loses track of time."
#     },
#     "4": {
#         "prompt": "<my-character> walking out of the library holding
# a book, sunset sky",
#         "caption": "With a new adventure in her hands, she heads
# home happy."
#     }
# }

# %% [markdown]
# ### Textual Inversion Input Generation

# %%
# Empty CUDA cache
torch.cuda.empty_cache()

# Log in to HuggingFace if needed...
login(token="YOUR_ACCESS_TOKEN") # Replace with an actual key

# Using the 'lykon/dreamshaper-xl-turbo' model to produce more
# photorealistic, anatomically accurate,
# and aesthetically pleasing human characters with consistent
# facial features and better detail, especially in full-body
# renderings.
pipeline =
    AutoPipelineForText2Image.from_pretrained('lykon/dreamshaper-xl-turbo',
                                                torch_dtype=torch.float16,
                                                variant="fp16")

pipeline.scheduler =
    DPMSolverMultistepScheduler.from_config(pipeline.scheduler.config)
pipeline = pipeline.to("cuda")

images_path = "character_images"
os.makedirs(images_path, exist_ok=True)

# Prompt engineering

```

```

full_body_prompt = f'{CHARACTER_PROMPT} , standing facing
    forward in plain space, full body and head, wide angle view of
    full body from head to feet, sharp focus, photorealistic'
full_body_negative = "cropped, half body, blurry, bad anatomy,
    misshapen face, blurry face"
face_prompt = f'{CHARACTER_PROMPT} , closeup of face,
    portrait'
face_negative = "blurry, bad anatomy, misshapen face, blurry
    face"

num_samples = 5
images = []
for i in range(num_samples):
    # Generate full body images
    image_1 = pipeline(full_body_prompt,
        num_inference_steps=50,
        guidance_scale=8,
        negative_prompt=full_body_negative
    ).images[0]
    # Generate facial images
    image_2 = pipeline(face_prompt,
        num_inference_steps=50,
        guidance_scale=8,
        negative_prompt=face_negative
    ).images[0]
    images.append(image_1)
    images.append(image_2)

grid = display_images_in_grid(images, 2, num_samples)

for i, img in enumerate(images):
    img.save(f'{images_path}/image_{i}.jpg')

del pipeline

# The input images will be saved in the 'character_images' folder

# %% [markdown]
# ### Using Textual Inversion to Learn the New Character

# %%
BASE_MODEL = "stabilityai/stable-diffusion-2"

```

```

# This is the token we will use to represent our new character in
    image generation prompts (I have used special characters
    "<",">" to differentiate this token from other real words)
CH_TOKEN = "<my-character>"

# %%
# Load Training Images...

images = []
for filename in os.listdir(images_path):
    if filename.lower().endswith(("jpg", "png", "jpeg")): # Check
        for file_image_format
            image_path = os.path.join(images_path, filename)
            images.append(Image.open(image_path).resize((512, 512)))

display_images_in_grid(images, 1, len(images))

# %%
# Textual Inversion Prompt Templates
templates = [
    "a DSLR photo of {}",
    "a casual photo of {}",
    "a candid photo of {}",
    "a close-up photo of {}",
    "a cropped photo of {}",
    "a fashion photo of {}",
    "a good photo of {}",
    "a high-resolution photo of {}",
    "a photo of {}",
    "a photo of {} in a studio",
    "a photo of {} in natural light",
    "a photo of {} looking at the camera",
    "a photo of {} with cinematic lighting",
    "a portrait of {}",
    "a professional photo of {}",
    "a realistic photo of {}",
    "a rendering of {}",
    "a rendition of {}",
    "a studio-lit photo of {}",
    "a picture of {}",
    "an image of {}",
    "an old photo of {}",
    "the photo of {}"
]

# %%

```

Setting Up the Dataset...

```
class CharacterTIDataset(Dataset):
    def __init__(
        self,
        dataset_path,
        tokenizer,
        size=512,
        repeats=100, # Use each image 100 times
        character_token="<my-character>",
    ):

        self.dataset_path = dataset_path
        self.image_paths = []
        for filename in os.listdir(self.dataset_path):
            if filename.lower().endswith(("jpg", "png", "jpeg")):
                # Check for file image format
                self.image_paths.append(os.path.join(self.dataset_path,
                                                       filename))

        self.num_images = len(self.image_paths)
        self.templates = templates

        self.tokenizer = tokenizer
        self.size = size
        self.repeats = repeats
        self.character_token = CH_TOKEN

    def __len__(self):
        return self.num_images * self.repeats

    def __getitem__(self, i):
        image = Image.open(self.image_paths[i %
            self.num_images]).convert("RGB")

        # Resize image
        image = image.resize((self.size, self.size),
                               resample=PIL.Image.BILINEAR)
        # Flip image horizontally (with 0.5 probability)
        image = transforms.RandomHorizontalFlip(p=0.5)(image)
        # Normalise the image's pixel values from the range [0,
            255] to [-1, 1]
        image = np.array(image).astype(np.uint8)
        image = ((image / (255/2)) - 1.0).astype(np.float32)
        # Convert the image into a PyTorch tensor
        image = torch.from_numpy(image).permute(2, 0, 1)
```

Pick a random text prompt from the templates

```
prompt_text =
    random.choice(self.templates).format(self.character_token)

dataset_item = {}
dataset_item["input_ids"] = self.tokenizer(
    prompt_text,
    padding="max_length",
    truncation=True,
    max_length=self.tokenizer.model_max_length,
    return_tensors="pt",
).input_ids[0]
dataset_item["pixels"] = image

return dataset_item
```

Loading the Tokenizer...

```
tokenizer = CLIPTokenizer.from_pretrained(
    BASE_MODEL,
    subfolder="tokenizer",
)
```

Add the new character token (to be trained) in tokenizer...

```
new_token_added = tokenizer.add_tokens(CH_TOKEN)
```

if not new_token_added:

```
    raise ValueError(f'Use a different character token! This
        tokenizer already contains {CH_TOKEN}.')
```

%%

Loading the Stable Diffusion Model...

```
text_encoder = CLIPTextModel.from_pretrained(
    BASE_MODEL,
    subfolder="text_encoder"
)
```

*# Resize token embeddings after adding the new character token in
 the tokenizer.*

*# This will create a new embedding vector for the new character
 token.*

```
text_encoder.resize_token_embeddings(len(tokenizer))
```

```
vae = AutoencoderKL.from_pretrained(
```

```

    BASE_MODEL,
    subfolder="vae"
)
UNET = UNet2DConditionModel.from_pretrained(
    BASE_MODEL,
    subfolder="unet"
)

# %%
# We can initialize the embedding for our new token with the one
# for "person" for a start...
person_token_id = tokenizer.encode("person",
    add_special_tokens=False)[0]

# Initialise the newly added character token with the embeddings
# of the word "person"
character_token_id =
    tokenizer.convert_tokens_to_ids(CH_TOKEN)
token_embeds =
    text_encoder.get_input_embeddings().weight.data
token_embeds[character_token_id] = token_embeds[person_token_id]

# %%
# Textual inversion involves only training the new embedding
# vector.
# All other model parameters should be frozen to avoid
# catastrophic forgetting.
# These other parameters will remain untouched by our training
# loop.

def freeze_parameters(parameters):
    for p in parameters:
        p.requires_grad = False

# Freeze all the VAE and UNet parameters
freeze_parameters(vae.parameters())
freeze_parameters(unet.parameters())

# Freeze all the Text Encoder parameters except token embeddings
freeze_parameters(text_encoder.text_model.encoder.parameters())
freeze_parameters(text_encoder.text_model.final_layer_norm.parameters())
freeze_parameters(text_encoder.text_model.embeddings.position_embedding.parameters())

# %%
# Create Noise Scheduler for Training...
noise_scheduler = DDPMSScheduler.from_pretrained(

```

```

    BASE_MODEL,
    subfolder="scheduler"
)

# %% [markdown]
# ##### Training

# %%
# Setting Up Hyperparameters...

LEARNING_RATE = 5e-05
NUM_TRAINING_STEPS = 4000
BATCH_SIZE = 4
GRAD_ACC_STEPS = 1
OUTPUT_DIR = "character-ti"

os.makedirs(OUTPUT_DIR, exist_ok=True)

# %%
# Whether or not to use the character description prompt while
# training
TRAIN_WITH_TEXT_PROMPT = True

# %%
# Training function

def training_function(text_encoder, vae, unet):
    batch_size = BATCH_SIZE
    gradient_accumulation_steps = GRAD_ACC_STEPS
    learning_rate = LEARNING_RATE
    max_train_steps = NUM_TRAINING_STEPS
    output_dir = OUTPUT_DIR

    accelerator = Accelerator(
        gradient_accumulation_steps=gradient_accumulation_steps,
        mixed_precision="fp16"
    )

    text_encoder.gradient_checkpointing_enable()
    unet.enable_gradient_checkpointing()
    # Prepare dataset and dataloader...
    dataset = CharacterTIDataset(
        dataset_path=images_path,
        tokenizer=tokenizer,
        size=vae.config.sample_size,

```

```

        character_token=CH_TOKEN,
        repeats=100,
    )
    train_dataloader = DataLoader(dataset, batch_size=batch_size,
                                  shuffle=True)

    # Scale the learning rate
    learning_rate = (learning_rate * gradient_accumulation_steps *
                     batch_size * accelerator.num_processes)

    # Initialize the optimizer
    optimizer = AdamW(
        text_encoder.get_input_embeddings().parameters(), # only
        optimize the embeddings
        lr=learning_rate,
    )

    text_encoder, optimizer, train_dataloader =
        accelerator.prepare(
            text_encoder, optimizer, train_dataloader
        )

    # Move VAE and UNet to device
    vae.to(accelerator.device, dtype=torch.float16)
    unet.to(accelerator.device, dtype=torch.float16)
    vae.eval()
    unet.train()

    num_train_epochs = math.ceil(max_train_steps /
                                   (math.ceil(len(train_dataloader) /
                                                gradient_accumulation_steps)))

    # Progress bar
    progress_bar = tqdm(range(max_train_steps), disable=not
                        accelerator.is_local_main_process)
    progress_bar.set_description("Steps")
    global_step = 0

    for epoch in range(num_train_epochs):
        text_encoder.train()
        for step, batch in enumerate(train_dataloader):
            with accelerator.accumulate(text_encoder):
                # Convert images to latent space
                latents =
                    vae.encode(batch["pixels"].to(dtype=torch.float16)).latent_dist.sample().detach()
                latents = latents * 0.18215

```

```

# Add noise to latents
noise = torch.randn_like(latents)
timesteps = torch.randint(0,
                           noise_scheduler.config.num_train_timesteps,
                           (latents.shape[0],),
                           device=latents.device).long() # random timestep
noisy_latents = noise_scheduler.add_noise(latents,
                                           noise, timesteps)

# Get the text embedding for conditioning
text_embedding =
    text_encoder(batch["input_ids"])[0]

# Predict the noise
noise_pred = unet(noisy_latents, timesteps,
                  text_embedding.to(torch.float16)).sample

# Get the target for loss depending on the
# prediction type
if noise_scheduler.config.prediction_type ==
    "epsilon":
    target = noise
elif noise_scheduler.config.prediction_type ==
    "v_prediction":
    target = noise_scheduler.get_velocity(latents,
                                           noise, timesteps)
else:
    raise ValueError(f"Unknown prediction type")

loss = F.mse_loss(noise_pred, target,
                  reduction="none").mean([1, 2, 3]).mean()

if TRAIN_WITH_TEXT_PROMPT:
    with torch.no_grad():
        # Get embedding for image caption e.x. "A
        # photo of {CH_TOKEN}"
        char_input_ids = batch["input_ids"]
        char_embedding =
            text_encoder(char_input_ids)[0]
        char_embedding_mean =
            char_embedding.mean(dim=1) # shape:
            (batch_size, hidden_dim)

        # Get embedding for the fixed character
        description

```

```

        target_ids =
            tokenizer(Character_PROMPT,
                return_tensors="pt").input_ids.to(accelerator.device)
        target_embedding =
            text_encoder(target_ids)[0]
        target_embedding_mean =
            target_embedding.mean(dim=1) # shape:
            (1, hidden_dim)

        # Compute cosine similarity between each
        # char_embedding and the fixed description
        embedding_loss = 1 - F.cosine_similarity(
            char_embedding_mean,
            target_embedding_mean.expand_as(char_embedding_mean)
        ).mean()

        loss = loss + (0.1 * embedding_loss)

    accelerator.backward(loss)

    # Zero out the gradients for all token embeddings
    # except the newly added character embeddings
    if accelerator.num_processes > 1:
        grads =
            text_encoder.module.get_input_embeddings().weight.grad
    else:
        grads =
            text_encoder.get_input_embeddings().weight.grad

    # Get the index for tokens that we want to zero
    # the grads for
    index_grads_to_zero =
        (torch.arange(len(tokenizer)) !=
         character_token_id)
    grads.data[index_grads_to_zero, :] =
        grads.data[index_grads_to_zero, :].fill_(0)

    optimizer.step()
    optimizer.zero_grad()

if accelerator.sync_gradients:
    # Update progress bar...
    progress_bar.update(1)
    global_step += 1

# Save embeddings every 500 steps
if global_step % 500 == 0:
    save_path = os.path.join(output_dir,
        f"learned_embeddings-step-{global_step}.bin")
    learned_embeddings =
        accelerator.unwrap_model(text_encoder).get_input_embeddings()
    learned_embeddings_dict = {CH_TOKEN:
        learned_embeddings.detach().cpu()}
    torch.save(learned_embeddings_dict, save_path)

logs = {"loss": loss.detach().item()}
progress_bar.set_postfix(**logs)

# Training completed!
if global_step >= max_train_steps:
    break

accelerator.wait_for_everyone()

# Save the whole pipeline...
if accelerator.is_main_process:
    pipeline = StableDiffusionPipeline.from_pretrained(
        BASE_MODEL,
        text_encoder=accelerator.unwrap_model(text_encoder),
        tokenizer=tokenizer,
        vae=vae,
        unet=unet,
    )
    pipeline.save_pretrained(output_dir)
    # Save trained embeddings separately
    save_path = os.path.join(output_dir, f"learned_embeddings.bin")
    learned_embeddings =
        accelerator.unwrap_model(text_encoder).get_input_embeddings().weight
    learned_embeddings_dict = {CH_TOKEN:
        learned_embeddings.detach().cpu()}
    torch.save(learned_embeddings_dict, save_path)

accelerate.notebook_launcher(training_function,
    args=(text_encoder, vae, unet), num_processes=1)

for parameter in unet.parameters():
    if parameter.grad is not None:
        del parameter.grad

```

```

for parameter in text_encoder.parameters():
    if parameter.grad is not None:
        del param.grad

torch.cuda.empty_cache()

# %% [markdown]
# ### Generating Comic-Strip

# %%
torch.cuda.empty_cache()

# Set up the Stable Diffusion Pipeline
pipeline = StableDiffusionPipeline.from_pretrained(
    OUTPUT_DIR,
    scheduler=DPMSolverMultistepScheduler.from_pretrained(OUTPUT_DIR,
        subfolder="scheduler"),
    torch_dtype=torch.float16,
).to("cuda")

# %%
torch.cuda.empty_cache()

os.makedirs('generated_images', exist_ok=True)
os.makedirs('comic_images', exist_ok=True)

negative_prompt = "blurry, low quality, low resolution, deformed,
    disfigured, mutated, extra limbs, missing limbs, broken
    anatomy, bad anatomy, bad proportions, cloned face, duplicate
    face, ugly, artifact, grainy, jpeg artifacts, watermark, text,
    logo, distorted, long neck, fused fingers, extra fingers,
    missing fingers, poorly drawn face, poorly drawn hands, bad
    hands, bad face"

#Generate Images...
generated_images = []
for prompt, caption in PROMPTS_AND_CAPTIONS.items():
    image = pipeline(prompt,
        num_inference_steps=40,
        guidance_scale=8,
        negative_prompt=negative_prompt
    ).images[0]
    image.save(f'generated_images/{prompt}.png')
    generated_images.append(image)
display_images_in_grid(generated_images, 1, NUM_PANELS)

# %%
# Image-to-Image Style Transfer
# Convert the photorealistic panel images into an image that looks
    hand-drawn...

def convert_to_art(image):
    # Load Stable Diffusion pipeline...
    pipeline =
        AutoPipelineForImage2Image.from_pretrained("runwayml/stable-diffusion-v1-5",
            torch_dtype=torch.float16)
    pipeline.scheduler =
        DPMSolverMultistepScheduler.from_config(pipeline.scheduler.config)
    pipeline.safety_checker = None

    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    pipeline.to(device)

# Load IP-Adapter
pipeline.load_ip_adapter(
    "h94/IP-Adapter",
    subfolder="models",
    weight_name="ip-adapter_sd15.bin"
)
pipeline.set_ip_adapter_scale(0.5)

# Load LoRA
pipeline.unet = PeftModel.from_pretrained(pipeline.unet,
    "ghibli_lora")
pipeline.unet.eval()

# Process Image
image = image.convert("RGB")

# Size 512X512 for SD...
image_sd = transforms.Resize((512, 512),
    interpolation=transforms.InterpolationMode.BILINEAR)(image)
image_sd = transforms.CenterCrop(512)(image)
# Size 224x224 for the IP-Adapter...
image_ip = transforms.Resize((224, 224),
    interpolation=transforms.InterpolationMode.BILINEAR)(image)
image_ip = transforms.CenterCrop(224)(image_ip)

```

```

image_ip =
    transforms.ToTensor()(image_ip).unsqueeze(0).to("cuda")

# Run the style transfer...
with torch.autocast("cuda"):
    image = pipeline(
        prompt="A Ghibli style scene", # Prompt contains
        "Ghibli style" so that generated image is of that
        style
        image=image_sd,
        ip_adapter_image=image_ip,
        num_inference_steps=40,
        strength=0.8, # decrease value if too much of the
        original image is lost
        guidance_scale=8.5,
        negative_prompt="blurry face"
    ).images[0]

del pipeline

return image

# %%
# Convert to hand-drawn art...
stylized_images = []
for image in generated_images:
    stylized_image = convert_to_art(image)
    stylized_image.save(f'comic_images/{prompt}.png')
    stylized_images.append(stylized_image)
display_images_in_grid(stylized_images, 1, NUM_PANELS)

# %%
# Assemble Comic Strip!!!

def create_comic_strip(images, captions, font_path="Melon
Camp.ttf", panel_width=512, font_size=30, padding=20,
gutter=20):
    font = ImageFont.truetype(font_path, font_size)

    num_panels = len(images) # NUM_PANELS
    cols = math.ceil(math.sqrt(num_panels))
    rows = math.ceil(num_panels / cols)

    # Estimate average image height after resizing
    aspect_ratios = [img.height / img.width for img in images]

```

```

resized_heights = [int(panel_width * ar) for ar in
    aspect_ratios]

# Text wrapping configuration
line_height = font.getbbox("A")[3] + 5
char_width = font.getbbox("A")[2] - font.getbbox("A")[0]
chars_per_line = (panel_width - 2 * padding) // char_width

wrapped_captions = []
max_caption_height = 0
for caption in captions:
    wrapped = textwrap.fill(caption,
        width=chars_per_line)
    lines = wrapped.count('\n') + 1
    wrapped_captions.append(wrapped)
    max_caption_height = max(max_caption_height, lines *
        line_height)

# Max panel height (based on tallest resized image)
max_image_height = max(resized_heights)
panel_height = max_image_height + max_caption_height + padding

# Total comic strip dimensions with padding
total_width = cols * panel_width + (cols - 1) * gutter + 2 *
padding
total_height = rows * panel_height + (rows - 1) * gutter + 2 *
padding

comic_strip = Image.new("RGB", (total_width, total_height),
    color="white")
draw = ImageDraw.Draw(comic_strip)

for idx, (img, caption) in enumerate(zip(images,
    wrapped_captions)):
    row = idx // cols
    col = idx % cols

    x = padding + col * (panel_width + gutter)
    y = padding + row * (panel_height + gutter)

    # Resize image to fit panel width
    aspect_ratio = img.height / img.width
    new_height = int(panel_width * aspect_ratio)
    resized_img = img.resize((panel_width, new_height),
        Image.LANCZOS)

    comic_strip.paste(resized_img, (x, y))

```

```
# Draw caption with centered alignment
for j, line in enumerate(caption.split("\n")):
    line_width = font.getbbox(line)[2] -
        font.getbbox(line)[0]
    text_x = x + (panel_width - line_width) // 2
    text_y = y + resized_img.height + 5 + j *
        line_height
    draw.text((text_x, text_y), line, fill="black",
        font=font)
```

```
return comic_strip

captions = [i["caption"] for i in PROMPTS_AND_CAPTIONS.values()]
comic = create_comic_strip(stylized_images, captions)
display(comic)
comic.save("comic.png")
```

A.2 File: StyleTransfer.py

```
# %% [markdown]
# # Style Transfer (Using LoRA)

# %% [markdown]
# ### Get the Image Dataset
#
# This is a dataset of ~1800 Studio Ghibli style hand-drawn
# images, which we will use to give our comics the same
# hand-drawn effect.
# Make sure to generate captions for these images using
# 'image_captioning.ipynb'

# %%
# Get the Studio Ghibli Style Images Dataset... and Unzip it

!wget
  https://github.com/TachibanaYoshino/AnimeGANv2/releases/download/1.0/Hayao.tar.gz
!tar -xzf Hayao.tar.gz

# %% [markdown]
# ### Imports and Initial Setup

# %%
# Manage Hex Cloud GPU Usage...
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "3"

# %%
# Import Libraries

import csv
import lightning.pytorch as pl
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from diffusers import AutoPipelineForImage2Image,
  DPMSolverMultistepScheduler, StableDiffusionPipeline
from peft import get_peft_model, LoraConfig,
  PeftModel
from PIL import Image
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms

# %% [markdown]
# ### LoRA Setup

# %%
CAPTIONS_FILE = "image_captions.csv"

# Dataset to handle the image-caption pairs...
class GhibliDataset(Dataset):
    def __init__(self, dataset_path):
        self.dataset_path = dataset_path

        self.image_paths = []
        for filename in os.listdir(self.dataset_path):
            if filename.lower().endswith(("jpg", "png", "jpeg")):
                # Check for file image format
                self.image_paths.append(os.path.join(self.dataset_path,
                    filename))
        self.image_paths = sorted(self.image_paths)

        with open(CAPTIONS_FILE, mode='r', encoding='utf-8') as
            csv_file:
                reader = csv.DictReader(csv_file)
                self.filename_to_caption = {row['filename'].strip():
                    row['caption'].strip() for row in reader}

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image_path = self.image_paths[idx]

        image = Image.open(image_path).convert("RGB")

        # Resize the image to 512x512
        image = transforms.Resize(512,
            interpolation=transforms.InterpolationMode.BILINEAR)(image)
        image = transforms.RandomCrop(512)(image)
        # Convert the image to a tensor
        image = transforms.ToTensor()(image)
        # Normalize the image with mean and std of 0.5
        image = transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5,
            0.5])(image)

        caption = self.filename_to_caption.get(image_path, "A
            Ghibli style scene")
        return image, caption
```

```

# %%
# Class to handle training the LoRA

class LoRATrainer(pl.LightningModule):
    def __init__(self, pipeline, lr, train_strength=0.5):
        super().__init__() # Initialize parent class

        self.unet = pipeline.unet
        self.vae = pipeline.vae
        self.tokenizer = pipeline.tokenizer
        self.text_encoder = pipeline.text_encoder
        self.scheduler = pipeline.scheduler
        self.train_strength = train_strength
        self.lr = lr

        self.prefixes = [
            "A Ghibli style image showing ",
            "A Ghibli style painting depicting ",
            "A Ghibli style scene of "
        ]

    def training_step(self, batch, batch_idx):
        images, captions = batch

        # Add a prefix to the captions to represent our style...
        captions = [np.random.choice(self.prefixes) + caption
                     for caption in captions]

        # Tokenize captions
        tokens = self.tokenizer(captions,
                                return_tensors='pt', padding=True,
                                truncation=True).input_ids.to(self.device)

        # Encode images to latents
        latents = self.vae.encode(images).latent_dist.sample() * 0.18215

        # Add noise
        noise = torch.randn_like(latents)
        timesteps = torch.randint(0, int(1000 *
                                         self.train_strength), (images.size(0),),
                                   device=self.device).long()
        noisy_latents = self.scheduler.add_noise(latents, noise,
                                                timesteps)

        # Predict noise
        noise_pred = self.unet(noisy_latents, timesteps,
                               encoder_hidden_states=self.text_encoder(tokens)[0])['sample']

        # Compute MSE loss
        loss = torch.nn.functional.mse_loss(noise_pred,
                                              noise)
        return loss

    def configure_optimizers(self):
        return torch.optim.AdamW(self.unet.parameters(),
                                   lr=self.lr)

# %% [markdown]
# ### Training

# %%
# Hyperparameters

BASE_MODEL = "runwayml/stable-diffusion-v1-5"
DATASET_PATH = "Hayao/style"
LORA_RANK = 4
LORA_ALPHA = 8
LORA_DROPOUT = 0.1
TRAIN_STRENGTH = 0.5
LEARNING_RATE = 1e-6
BATCH_SIZE = 4
GRAD_ACC_STEPS = 4
NUM_EPOCHS = 100
OUTPUT_DIR = "ghibli_lora"

# %%

torch.cuda.empty_cache()

pipeline = StableDiffusionPipeline.from_pretrained(BASE_MODEL)
pipeline.scheduler = DPMSolverMultistepScheduler.from_config(pipeline.scheduler.config)
pipeline.safety_checker = None # Disable for creative outputs

# Inject LoRA using the PEFT Library
lora_config = LoraConfig(
    r=LORA_RANK,
    lora_alpha=LORA_ALPHA,

```

```

init_lora_weights='gaussian',
target_modules=['to_k', 'to_q', 'to_v', 'to_out.0',
                'ResnetBlock2D', 'CrossAttention'],
lora_dropout=LORA_DROPOUT,
bias='none'
)
pipeline.unet = get_peft_model(pipeline.unet, lora_config)

```

```

# Prepare Dataloader
dataset = GhibliDataset(DATASET_PATH)
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE,
                        shuffle=True, num_workers=4)

```

```

# Train!
model = LoRATrainer(pipeline=pipeline, lr=LEARNING_RATE,
                    train_strength=TRAIN_STRENGTH)
trainer = pl.Trainer(
    max_epochs=NUM_EPOCHS,
    precision='16-mixed',
    accumulate_grad_batches=GRAD_ACC_STEPS
)
trainer.fit(model, train_dataloaders=dataloader)

```

```

# Save
os.makedirs(OUTPUT_DIR, exist_ok=True)
model.unet.save_pretrained(OUTPUT_DIR)

```

```

# %% [markdown]
# ### Test Inference

```

```

# %%
# Text-to-Image Generation

```

```
PROMPT = "A girl with golden hair walks into the forest."
```

```

pipeline =
    StableDiffusionPipeline.from_pretrained(BASE_MODEL)
pipeline.scheduler =
    DPMSolverMultistepScheduler.from_config(pipeline.scheduler.config)
pipeline.unet = PeftModel.from_pretrained(pipeline.unet,
    OUTPUT_DIR)
pipeline.safety_checker = None
pipeline = pipeline.to('cuda')

```

```

# Add "A Ghibli style scene of" to trigger generation in the
trained art-style...

```

```
ghibli_prompt = f"A Ghibli style scene of {PROMPT}"
```

```

image = pipeline(prompt=ghibli_prompt,
                 num_inference_steps=40,
                 guidance_scale=8
                 ).images[0]

```

```
display(image)
```

```

# %%
# Image-to-Image Style Transfer

```

```
IMAGE_PATH = "image.png"
```

```

# Load Stable Diffusion pipeline...
pipeline =
    AutoPipelineForImage2Image.from_pretrained(BASE_MODEL,
    torch_dtype=torch.float16)
pipeline.scheduler =
    DPMSolverMultistepScheduler.from_config(pipeline.scheduler.config)
pipeline.safety_checker = None
pipeline = pipeline.to(device)

```

```

# Load IP-Adapter
pipeline.load_ip_adapter(
    "h94/IP-Adapter",
    subfolder="models",
    weight_name="ip-adapter_sd15.bin"
)
pipeline.set_ip_adapter_scale(0.5)

```

```

# Load LoRA
pipeline.unet = PeftModel.from_pretrained(pipeline.unet,
    OUTPUT_DIR)
pipeline.unet.eval()

```

```

# Load and Process Image
image = Image.open(image_path).convert("RGB")
# Size 512X512 for SD...
image_sd = transforms.Resize((512, 512),
    interpolation=transforms.InterpolationMode.BILINEAR)(image)
image_sd = transforms.CenterCrop(512)(image)
# Size 224x224 for the IP-Adapter...
image_ip = transforms.Resize((224, 224),
    interpolation=transforms.InterpolationMode.BILINEAR)(image)

```

```
image_ip = transforms.CenterCrop(224)(image_ip)
image_ip = transforms.ToTensor()(image_ip).unsqueeze(0).to("cuda")
```

```
# Run the style transfer...
with torch.autocast("cuda"):
    image = pipeline(
        prompt="A Ghibli style scene", # Prompt contains "Ghibli
            style" so that generated image is of that style
```

```
image=image_sd,
ip_adapter_image=image_ip,
num_inference_steps=40,
strength=0.95,
guidance_scale=8.5,
negative_prompt="blurry face"
    ).images[0]
display(image)
```

Appendix B

User Study

Displayed below is the survey that was carried out to evaluate the comics generated by this project.

Enhancing Character Consistency in AI-Generated Comics via Textual Inversion (FYP)

Hello there! Thanks for taking the time to complete this study for my final year individual project.

Please complete the participation consent form below before moving on to the study. For more information, please take a look at the participant information sheet here - <https://drive.google.com/file/d/1VYXYikiHGak0dVMbBg0Z5UhGf0pXPTOp/view?usp=sharing>

* Indicates required question

1. I have been provided with information explaining what participation in this project involves. [?]

Mark only one oval.

☐ Yes

2. I have had an opportunity to ask questions and discuss this project (if needed). I have received satisfactory answers to any questions I have asked.

Mark only one oval.

☐ Yes

3. I have received enough information about the project to make a decision about my participation.

Mark only one oval.

☐ Yes

4. I understand that I am free to withdraw my consent to participate in the project at any time without having to give a reason for withdrawing.

Mark only one oval.

☐ Yes

5. I understand the nature and purpose of the procedures involved in this project. These have been communicated to me on the information sheet accompanying this form.

Mark only one oval.

☐ Yes

6. I understand and acknowledge that the investigation is designed to promote scientific knowledge and that the University of Bath will use the data I provide only for the purpose(s) set out in the information sheet.

Mark only one oval.

☐ Yes

7. I understand the data I provide will be treated as confidential, and that on completion of the project, my name or other identifying information will not be disclosed in any presentation or publication of the research.

Mark only one oval.

☐ Yes

8. I understand that my consent to use the data I provide is conditional upon the University complying with its duties and obligations under current data protection legislation

Mark only one oval.

☐ Yes

Below is an AI-generated comic strip and the prompt used to generate it. Answer the following questions based on the image displayed. (1=Poor, 10=Excellent)

A comic strip for "Sarah visits the library"



9. Rate the image in terms of "Character Consistency across Panels" (eg., change in facial features, hair, outfit)

Mark only one oval.

1 2 3 4 5 6 7 8 9 10
☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

10. Rate the image in terms of "Story Coherence" (i.e. the story makes sense)

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

11. Rate the image in terms of "Adherence to the Prompt" *

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

12. Rate the image in terms of "Image quality" *

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

13. Please provide an explanation for your choices above.

Below is an AI-generated comic strip and the prompt used to generate it. Answer the following questions based on the image displayed. (1=Poor, 10=Excellent)

A comic strip for "Sarah visits the library"



14. Rate the image in terms of "Character Consistency across Panels" (eg., change in facial features, hair, outfit)

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

15. Rate the image in terms of "Story Coherence" (i.e. the story makes sense)

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

16. Rate the image in terms of "Adherence to the Prompt" *

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

17. Rate the image in terms of "Image quality" *

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

18. Please provide an explanation for your choices above.

Below is an AI-generated comic strip and the prompt used to generate it. Answer the following questions based on the image displayed. (1=Poor, 10=Excellent)

Sarah visits the library



Sarah steps into the library, ready for a quiet afternoon.



She scans the shelves and finds a book that catches her eye.



Engrossed in the story, she loses track of time.



With a new adventure in her hands, she heads home happy.

19. Rate the image in terms of "Character Consistency across Panels" (eg., change in facial features, hair, outfit)

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

20. Rate the image in terms of "Story Coherence" (i.e. the story makes sense)

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

21. Rate the image in terms of "Adherence to the Prompt". *

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

22. Rate the image in terms of "Image quality" *

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

23. Please provide an explanation for your choices above.

Below is an AI-generated comic strip and the prompt used to generate it. Answer the following questions based on the image displayed. (1=Poor, 10=Excellent)

Sarah visits the library



Sarah steps into the library, ready for a quiet afternoon.



She scans the shelves and finds a book that catches her eye.



Engrossed in the story, she loses track of time.



With a new adventure in her hands, she heads home happy.

24. Rate the image in terms of "Character Consistency across Panels" (eg., change in facial features, hair, outfit)

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

28. Please provide an explanation for your choices above.

25. Rate the image in terms of "Story Coherence" (i.e. the story makes sense)

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

26. Rate the image in terms of "Adherence to the Prompt". *

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

27. Rate the image in terms of "Image quality" *

Mark only one oval.

1	2	3	4	5	6	7	8	9	10
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

This content is neither created nor endorsed by Google.

Google Forms